

Student lab guide - Threat Modeling and Secure Code Review

Field Research Exchange (FRX) two-day practicum

- How to use this guide
- Required starting material
- Local setup and handling
- Record discipline
- Day 1 — Threat Modeling Practicum
 - One retained review
 - Handling and decision boundary
- Lab 1 — System and asset model
 - Assignment and inputs
 - Procedure
 - Checkpoints
- Lab 2 — Threat scenarios
 - Assignment and preparation
 - Worked partial answer — T1
 - Procedure
 - Checkpoints
- Lab 3 — Risk and Response
 - Assignment and inputs
 - Worked partial answer — retain T1
 - Procedure
 - Checkpoints
- Lab 4 — Peer model review
 - Assignment and inputs
 - Worked partial review — PR1
 - Review tests
 - Procedure
 - Checkpoints
- Day 2 student lab guide — Secure Code Review Practicum
 - Purpose
 - Day 2 sequence
 - Starting evidence and working files
 - Handling notice
 - Package setup

- Day 2 Lab 1 — Threat-informed review plan
 - Assignment and inputs
 - Worked partial plan — CRP1 / PTH1
 - Procedure
 - Checkpoints
- Day 2 Unit 2 guided practice — Strategy and crawl handoff
 - Purpose and starting material
 - Worked partial FRX strategy and crawl — PTH1
 - Procedure
 - Checkpoints
- Day 2 Unit 3 guided practice — Python source-reading handoff
 - Purpose and starting material
 - Worked partial FRX source trace — PTH1
 - Procedure
 - Checkpoints
- Day 2 Lab 2 — Manual Code Crawl-informed trace
 - Assignment and inputs
 - Worked partial evidence — PTH1
 - EV1 — source observation
 - EV2 — runtime observation
 - CF1 — candidate finding
 - Observation choices
 - Procedure
 - Checkpoints
- Day 2 Lab 3 — JWAC checklist evidence pass
 - Outcome and working conditions
 - What the four statuses mean
 - Worked FRX partial answer — RS1
 - Learner assignment
 - Steps
 - Checkpoints
 - Completion condition
- Lab 4 — Finding and supply-chain validation
 - Outcome and retained work
 - Handling and decision boundaries
 - Starting material
 - Intentionally shared FRX partial — SV1 and FN1
 - Six lab steps
 - Checkpoints
 - Completion and reflection

- Lab 5 — Remediation review
 - Outcome and retained work
 - Handling and decision boundaries
 - Starting material
 - Intentionally shared FRX partial — RM1 for FN1 / PCH-001
 - Six lab steps
 - Checkpoints
 - Completion and reflection
- Lab 6 — Delegate briefing
 - Outcome and retained work
 - Decision and handling boundaries
 - Starting material
 - Intentionally shared FRX partial — RM1 becomes one DB1 decision line
 - Six lab steps
 - Checkpoints
 - Submit
- Field Research Exchange — existing-system review packet
 - Read the assignment
 - Evidence status and owners
 - People and records
 - Follow the ordinary workflow
 - Interfaces and effects
 - Data and system responsibilities
 - Questions with evidence owners
- From system documentation to a threat-model diagram
 - Worked partial answer: report read
 - Context, detail, and a readability check
- Supplied FRX diagram collection
 - Generic DFD example
 - Generic threat-analysis DFD
 - FRX system context
 - FRX implementation architecture
 - FRX report-read sequence
 - FRX sign-in detail
 - FRX search/read detail
 - FRX document-intake sequence
 - FRX analytics-fetch sequence
 - FRX analytics-import sequence
 - FRX data model
 - FRX report-read model fragment

- Threat discovery reference — STRIDE, OWASP and CWE
 - Read a DFD as an investigation map
 - Apply STRIDE to a specific element or flow
 - Use OWASP Top 10:2025 to broaden the mechanism
 - Choose a CWE only when its mechanism fits
 - PASTA in context
- Risk and Response reference
 - Build a risk argument before selecting a response
 - Workshop reporting matrix
 - Responses and verifiable requirements
 - DREAD: recognize the method and its limits
- FRX source orientation for Day 2
 - Review target
 - Repository responsibilities
 - Route-to-source index
 - Day 1 handoff seed
 - Python and Flask construct key
 - Unit 3 source-reading boundary
 - Evidence discipline
- Unit 5 current-criteria reference
 - Checklist status vocabulary
 - Current criteria and precise mappings
 - How to reconcile a local rule with current guidance
 - Sources
- FRX frozen pipeline evidence bundle
 - Bound subject
 - Artifacts
 - Integrity check
 - Handling notice
- FRX remediation-review change catalog
 - Purpose and limits
 - PCH-001 — Runtime session-signing key — worked partial
 - PCH-002 — Login query, password storage, and audit handling
 - PCH-003 — Report object authorization
 - PCH-004 — Analytics destination and object authorization
 - PCH-005 — Audit and error-output correction — extension

Distribution format. Keep one FRX review record across both days. Use the editable Day 1 and Day 2 DOCX workbooks in `1abs/`. The evidence packet, notation lesson, diagrams, and later reference sheets are included as appendices in this PDF.

How to use this guide

Keep one Field Research Exchange (FRX) review record across both days. Day 1 establishes the system model, threats, risk responses, verification questions, and peer-reviewed implementation handoff. Day 2 retains those identifiers while planning and executing the code review, governing evidence, verifying remediation, and preparing a bounded delegate recommendation.

The workshop contains ten formal or guided work segments:

- **Day 1:** four formal labs—system and asset model; threat scenarios; risk and response; peer model review.
- **Day 2:** six formal labs—review plan; manual crawl; governed checklist pass; signal validation; remediation review; delegate briefing—plus guided practice in review strategy and Python source reading.

During each exercise, record assistance, recovery, information gaps, and supported differences from the examples so the retained review record shows how conclusions were reached.

Required starting material

- The supplied FRX learner project under `frx-project/` and learner evidence under `materials/`.
- The Day 1 and Day 2 record structures referenced in each section.
- The authorized code-review checklist and APPROVE template when the instructor introduces them. These controlled inputs are supplied separately and should not be copied into learner work.

Local setup and handling

Use PowerShell 7.4 or later from the extracted package root:

```
./scripts/setup.ps1
./scripts/verify-environment.ps1
```

The FRX application contains deliberate weaknesses and synthetic data for local defensive training. Keep services on loopback. Do not enter operational credentials or information, connect operational systems, add public targets, or expose the application publicly. Follow the instructor's start/reset direction and the specific evidence boundaries in each lab.

Record discipline

Requirements, documented claims, observed behavior, unresolved questions, model records, manual evidence, candidate findings, checklist status, tool-signal dispositions, confirmed findings, remediation proof, remaining risk, delegate recommendation, and authorized organizational decisions are distinct. Retain links and revision history instead of silently rewriting an earlier record to match a later conclusion.

Day 1 — Threat Modeling Practicum

One retained review

Use one working document named **FRX review** throughout Day 1. Begin with the [existing-system evidence packet](#), [diagram collection](#), and **record structures** ([labs/day-1-record-templates.docx](#)). Retain stable identifiers and revise the same artifact rather than recopying records between labs.

Work in pairs unless the instructor states that a documented substitution is necessary.

Lab	Starting point	Retained output
1 — System and asset model	Supplied FRX evidence, diagrams, and worked report-read fragment	SCOPE-1, assets, DFD, flow key, boundaries, and questions
2 — Threat scenarios	Retained Lab 1 model and T1 partial example	Causal scenarios, mappings, uncertainty, and coverage
3 — Risk and Response	Retained scenario register and T1/R1/V1 partial example	Risk arguments, responses, requirements, verification, and residual risk
4 — Peer model review	Complete Lab 1–3 artifact and PR1 partial review	Review comments, dispositions, reconciled artifact, and Day 2 handoff

Handling and decision boundary

Use only the supplied local application, course identities, and records. Do not enter operational information, expose the application publicly, or contact external targets. The exercises produce review work products and recommendations; they do not establish implementation behavior, checklist status, application approval, or certification authority.

If a supported conclusion differs from a worked example, keep it and record the evidence and assumptions. Counts bound practice; they do not prove sufficient review coverage.

Lab 1 — System and asset model

Assignment and inputs

Produce a model another analyst can use to investigate the existing FRX application. Work in pairs: one person traces evidence, the other records; switch after Checkpoint 1A. Paper, a text editor, or a diagramming tool are equally valid. You do not need to write Mermaid or change code.

Use the [system evidence packet](#), [supplied diagram collection](#), and [notation lesson and worked fragment](#). Follow the handling notice in the packet. Prerequisites are HTTP, basic authentication/authorization concepts, and reading simple diagrams; Python expertise is unnecessary.

Before starting, explain the difference between a signed-in identity and permission to read a particular report. If you cannot, revisit evidence E2 and the worked boundary explanation.

Create one working document called **FRX review**. Keep it through all four labs. Use the sections Scope, Assets, Model, Questions, Scenarios, Risk and Response, and Peer Review. You may paste the **blank record structures** ([labs/day-1-record-templates.docx](#)). Only fill the first four sections now.

Procedure

1. **Trace the normal workflow.** Input: packet E6, demonstration, and the report-read/intake/analytics sequences. Follow sign-in → search Prairie → report 101 → score → upload → sign-out. Record one observed request, returned value and persistent effect, tagging each as demo observation or supplied engineering evidence. **Check:** distinguish the report's database row from the separate file receipt; do not assume that selecting a report associates the uploaded file with it.
2. **State the scope.** Input: packet E1–E3 and architecture view. Write the decision being supported, the application version/baseline described by the packet, the people and operations in scope, and external dependencies. Name the browser/client explicitly. Identify one excluded implementation and its remaining dependency. **Result:** scope SCOPE-1 with evidence links. **Check:** an analyst can tell whether an API-only route, file store, provider interaction, and managed registration belong in the review.
3. **Identify assets.** Input: E1, E2, E4, E5 and ER view. Retain worked asset A1, then add data and supporting assets implicated by the other paths. For each, record ID, data/location or service, business/evidence owner, confidentiality/integrity/availability need, consequence, and evidence. Mark an inferred owner as proposed. **Check:** include data in storage, in transit, and in use; credentials/session material and service availability matter as well as report contents. Complete Checkpoint 1A.
4. **Extend the worked DFD.** Input: P1, D1, entity E1 and flows F1–F4 from the notation lesson. Retain those IDs if you retain their meanings. Add processes, stores and external entities for remaining in-scope functions. Use a context view plus detail views if one page becomes crowded. **Result:** a readable diagram and element key with purpose and evidence. **Check:** every route in the packet has a place, including API-only paths; document any deliberate grouping. Do not invent a report-edit, file-download or assignment endpoint.
5. **Name the data flows.** Input: each supplied sequence and the route/effect table. Add directed request, response, lookup, write and event flows with stable F IDs. Label the data, not just “HTTP.” Record the source/destination and evidence in a flow key if the picture needs shorter labels. **Check:** trace one response as well as its request; the model must show what leaves the service and which store actually changes.

6. **Justify boundaries and questions.** Input: your flows, packet Q1–Q5 and worked TB1. Draw boundaries where control of input, authority or responsibility changes. Record each TB ID, crossing flows, reason, and which control remains unverified. Retain or refine supplied questions with an evidence owner and the decision their answer could change. **Check:** a box around a component is not proof of a deployed firewall or authorization check. Separate local topology from a proposed operating deployment.
7. **Reconcile a second workflow.** Input: your model and the intake or analytics sequence, choosing a path other than report read. Have your partner narrate the request and result without your help. Record an omission you corrected or a supported reason for retaining the model. **Result:** dated model revision, flow trace, and open questions. Complete Checkpoint 1B; retain this same artifact for Lab 2.

Checkpoints

1A — Scope and asset inventory, after Step 3. Show the instructor SCOPE-1 and your asset entries. Point to evidence for one access rule and one operational consequence. The inventory should cover the modeled data and supporting services; there is no minimum row count that proves completeness. If an owner or requirement is unknown, identify who can resolve it. Instructor feedback should identify missing evidence or an unsupported inference before diagram extension.

1B — Traceable model, after Step 7. Show the diagram, element/flow keys, boundary reasons and questions. A peer must trace the chosen workflow end to end and distinguish a file write, database read and provider exchange. Each in-scope route must be represented or deliberately grouped with a reason. Identify at least one unverified control and its evidence owner. A sound alternate decomposition is acceptable.

Success means that another analyst can locate relevant data, authority changes and uncertainties from your work. Appearance and completed boxes alone do not establish sufficient coverage. Keep unfinished coverage explicit for Lab 2 rather than silently declaring the system reviewed.

Lab 2 — Threat scenarios

Assignment and preparation

Use your retained Lab 1 model to produce a scenario register ready for risk decisions. Work in the same pair and switch recorder midway. Inputs: your scope/model/assets/questions, the [FRX evidence packet](#), [threat reference](#), and **record structures** (`labs/day-1-record-templates.docx`). If Lab 1 has a coverage gap, mark it and repair the relevant path before developing its scenario.

Recall: a threat is a possible adverse event; a weakness is a mechanism that may enable it; a confirmed vulnerability requires supporting evidence. You are developing hypotheses, not asserting that all prompts describe FRX defects.

Worked partial answer — T1

Evidence: E2 limits Alex to owned reports; E6 demonstrates the caller supplying a report ID. **Path:** entity E1 → F1 → P1 → D1 → F4; TB1; asset A1. **Actor/precondition:** a signed-in partner who can select a report ID.

Action and hypothesized failure: selects another partner's report; if the portal fails to check entitlement to that record, it returns the observation. **Consequence:** disclosure of another partner's report, contrary to E2, exposing observations described in E1.

Primary STRIDE: Information disclosure, because the consequence is an unauthorized read. Changing the request is a means; it does not by itself establish stored-data tampering. **OWASP prompt:** A01:2025 Broken Access Control. **Candidate CWE:** CWE-639 fits an authorization bypass using a caller-controlled record key; it remains conditional on that mechanism. **Uncertainty:** Q1 remains unresolved; normal read success is not evidence of the hypothesized denial failure.

Retain T1 and its links. You must add scenarios on other surfaces and examine coverage; merely relabeling T1 six ways is not six independent scenarios.

The worked A1 means report data across the collection. If your Lab 1 inventory uses a separate asset for each report, link T1 to the other partner's affected report asset in your own model. Keep your IDs and record the correspondence; do not relabel an unrelated asset just to copy the example.

Procedure

1. **Select a path.** Input: Lab 1 DFD and flow key. Choose an element or boundary-crossing flow other than T1's direct report read. Trace its source, destination, data and effect. **Record:** selected IDs and the evidence supporting that path. **Check:** the hypothesized action must be possible within the modeled interface, not a nonexistent report-edit function.
2. **Apply STRIDE.** Input: the six prompts in the threat reference. Ask each question about the selected path, including returned data and dependencies. **Record:** applicable questions, candidate threats, or a reason a prompt does not fit. **Check:** distinguish identity misuse, changed data, lost attribution, disclosure, unavailable work and greater privilege.
3. **Broaden the mechanism.** Input: the Top 10:2025 table. Consult relevant categories for missed mechanisms such as query/output interpretation, authorization or exceptional conditions. **Record:** a category and why it helped, or no useful additional mapping. **Check:** do not treat a category label as a complete threat or an exhaustive checklist.

4. **Write the causal scenario.** Input: one promising question and packet evidence. Assign T2 onward. Write actor/event, precondition, action/path, hypothesized failure, asset and consequence. **Record:** model IDs plus evidence and assumptions in the scenario structure. **Check:** another analyst can explain how the event causes the consequence and what missing evidence could disprove it. Complete Checkpoint 2A on your first new scenario.
5. **Refine the weakness mapping.** Input: curated CWE descriptions. Choose an entry if its mechanism matches; explain the fit and unresolved premise. Otherwise identify the evidence needed to select one. **Check:** do not infer missing authorization, code execution or unlimited outbound access simply from an interface name.
6. **Repeat and remove duplicates.** Input: remaining model surfaces. Aim for four student-authored scenarios in addition to T1 across distinct paths. Merge scenarios with the same actor/path/failure/consequence, retaining useful category links. **Result:** a concise register; use fewer records with an explicit unfinished-work note if time runs out. Count is a practice bound, not proof of completeness.
7. **Review coverage.** Input: your whole route/flow key and STRIDE notes. Mark each surface as covered by a scenario, considered with a reason, or still requiring investigation. Include API-only, event/error and dependency paths. **Record:** the highest-value unanswered question for risk discussion, without yet filling risk ratings or mitigations. Complete Checkpoint 2B and carry the register into Lab 3.

Checkpoints

2A — Causal scenario, after Step 4. Show the instructor a new record. Point to the route, affected asset, evidence and hypothesized control failure. If the instructor cannot trace cause to consequence, revise it before multiplying records.

2B — Register and coverage, after Step 7. A peer can follow each T ID to the model and distinguish evidence from hypothesis. Mappings are justified or explicitly unresolved. All six STRIDE categories were considered; omissions in review-surface coverage have a reason or an owner. Keep different defensible scenarios; do not force equal category counts.

Lab 3 — Risk and Response

Assignment and inputs

Develop response records that an engineering owner could implement and a reviewer could verify. Work in pairs. Retain Lab 2 scenario IDs and Lab 1 evidence; use the [risk reference and matrix](#) and **record structures** (`labs/day-1-record-templates.docx`). Do not rescore using DREAD; its brief illustration supplies method context only.

Worked partial answer — retain T1

Risk judgment: Medium likelihood: the caller selects a record on a reachable authenticated interface, but entitlement enforcement is unresolved. High impact: the hypothesized result is unauthorized partner-report disclosure under E1/E2. The practice matrix gives **High**, with limited confidence pending Q1. This is a priority for validation and response planning, not proof of an exploitable flaw.

Response: mitigate through server-side entitlement enforcement. Alternative: suspend partner report access pending review; that reduces exposure but interrupts the legitimate workflow. Recommend mitigation with prompt validation, and ask the owner to consider temporary access restriction if the uncertainty cannot be resolved acceptably. Hiding report links is insufficient because the caller supplies the ID at the service boundary.

R1: On every report read, search result, and report-related analytics request, the portal shall use the authenticated subject and current ownership/assignment/administrator policy to release only permitted report data. Denied requests shall not return protected report fields. Owner: engineering; E2 policy confirmation: coordinator. Basis: ASVS v5.0.0-8.2.2 and v5.0.0-8.3.1, explained in the risk reference.

V1: With the supplied identities and unchanged fixtures, verify Alex may read 101, Jordan may read 102, Riley may read assigned 103, and the administrator may read all three. Verify Alex cannot read 102 and Riley cannot read unassigned 102; check response bodies as well as denial status. Search results must respect the same policy. An unauthenticated request must release no report fields. Test the analytics path's policy separately; fixing direct read alone does not establish R1. These are proposed expectations, not observations that the baseline passes.

Residual risk and next action: correct authorization does not eliminate compromised legitimate accounts, excessive administrator access, or already disclosed copies. Engineering supplies route/test evidence; the coordinator confirms E2; the service owner decides any interim restriction. Revisit the rating when Q1 is answered. Keep T1 and add the risk/response fields to it.

Procedure

1. **Select records.** Input: Lab 2 register and coverage notes. Retain T1 and select two of your own scenarios for detailed treatment. **Record:** selection reasons based on consequence, uncertainty and review relevance. **Check:** the three records cover more than one underlying mechanism; counts are practice bounds.
2. **Rate with evidence.** Input: scenario preconditions, E1–E5 and the matrix. State likelihood and impact separately, distinguishing supported facts from assumptions. **Record:** matrix result, confidence and one

piece of evidence that could change each rating. **Check:** do not call likelihood High merely because a weakness is imaginable. Complete Checkpoint 3A for one learner-authored record.

3. **Compare priorities.** Input: the selected records. Explain relative order and any tie. **Record:** which evidence collection or response should happen first and why. **Check:** urgency can include uncertainty; the highest numeric-looking label is not an explanation.
4. **Choose a response.** Input: causal mechanism and business constraint. Select mitigate, avoid, transfer/share or recommend explicit acceptance. Compare at least two plausible options for one learner-authored scenario. **Record:** choice, tradeoff, accountable owner and any decision escalation. **Check:** the response changes the mechanism or its consequence; “use security best practices” is not actionable.
5. **Write required behavior.** Input: chosen response. Assign R2 onward and state actor/resource, trigger, required/forbidden behavior and failure handling. **Record:** policy/evidence basis and owner. **Check:** distinguish the requirement from a specific library choice and from a control already proven to work.
6. **Design verification.** Input: each R record. Assign V2 onward. Define setup, action and allowed/denied/failure observations that could support or disprove it. **Check:** include a legitimate workflow that must remain usable and inspect effects/body data, not just a successful status code. Link V to R and T without recopying them.
7. **State what remains.** Input: proposed response and unverified assumptions. Record residual risk, outstanding evidence, owner and next action. **Result:** linked risk/response records ready for peer review. Complete Checkpoint 3B. Do not describe a proposed mitigation as implemented or verified.

Checkpoints

3A — Supported rating, after Step 2. Show a learner-authored scenario, the separate likelihood/impact arguments and the correct matrix cell. Explain a plausible fact that would change the result. Revise unsupported certainty before proceeding.

3B — Response and proof, after Step 7. The instructor or peer can follow $T \rightarrow R \rightarrow V$, identify the responsible owner and see how verification could fail. Show the option comparison, legitimate workflow, residual risk and next action. Defensible alternate ratings are acceptable when assumptions and consequences support them.

Lab 4 — Peer model review

Assignment and inputs

Exchange your retained Lab 1–3 artifact with another pair. Use the system packet, supplied sequences, threat/risk references, and the **review record structure** (`labs/day-1-record-templates.docx`). You are assessing supported reasoning, coverage and verifiability. Do not introduce defects merely to have something to correct.

Worked partial review — PR1

Supplied incomplete claim: “T1 is mitigated by hiding other partners' report links. V1 checks that the links disappear.”

Location: T1 response and V1. **Evidence:** the report-read sequence shows a caller-supplied report ID at the portal boundary. **Issue/consequence:** hiding links does not establish service authorization; a request can still reach the route, so unauthorized disclosure remains plausible. **Requested action:** replace or supplement the UI measure with trusted-service entitlement checks and allowed/denied request observations.

Author disposition: revise R1/V1 to cover the service boundary and legitimate/denied access. Retain UI filtering as a usability aid, not the security proof. If the original record already requires those checks, retain it and explain where they appear rather than making a needless change.

Review tests

Read the exchanged artifact independently before asking the author to explain it. Apply these tests to the artifact as written:

- **Target:** is the baseline or revision, scope, evidence set, and declared unfinished work clear?
- **Model:** do named flows, identities, boundaries, stores, responses, and persistent effects agree with the supplied system evidence?
- **Coverage:** does each selected surface have a linked scenario, an explained exclusion, or an explicit open inquiry?
- **Decision chain:** can you follow model IDs → T → risk judgment → response/R → V → residual risk and owner without inventing a missing link?
- **Response and proof:** does required behavior interrupt the hypothesized failure while preserving legitimate work, and can allowed plus denied/failure observations expose an incorrect implementation?
- **Feedback:** does each material PR record name the target location, evidence, issue or supported confirmation, consequence, requested action, and observable completion condition?

A supported confirmation is a valid review result when the reasoning is sound. Do not manufacture a defect, force your preferred diagram layout, or replace a supported rating merely because your judgment differs.

Procedure

1. **Establish the review target.** Input: exchanged artifact. Record its baseline/revision, scope and any declared unfinished work. **Check:** both teams agree which version is being reviewed; request missing

input before guessing.

2. **Trace flows.** Input: peer model and supplied system sequences. Trace report access and one other path, including responses and persistent effects. **Record:** omissions, unsupported connections or a supported confirmation. **Check:** distinguish file intake from report registration and analytics fetch from import.
3. **Review scenario coverage.** Input: model/route key and scenario register. Follow at least two scenario links and inspect the coverage notes for omitted surfaces/categories. **Record:** evidence for a gap or a reason the coverage is defensible. **Check:** a scenario's mechanism and consequence follow from its assumed path; mappings are not decorative labels.
4. **Challenge response and verification.** Input: at least two $T \rightarrow R \rightarrow V$ chains, including a learner-authored one. Ask whether the response interrupts the mechanism and the test could expose failure without blocking all legitimate work. **Record:** any unsupported rating, missing owner, residual risk or untested path. Complete Checkpoint 4A before returning comments.
5. **Write actionable feedback.** Input: your review notes. Assign PR IDs; identify location, evidence, consequence and requested action. Prioritize material issues. If the work is sound, record a specific supported confirmation and the remaining limits. **Check:** the author can act without guessing what "improve security" means.
6. **Reconcile the returned review.** Input: comments on your own artifact. For each material comment, revise, retain with rationale, or escalate to a named evidence/decision owner. **Record:** disposition and updated record IDs/revision. **Check:** disagreement is allowed; ignoring the comment is not a disposition.
7. **Prepare the handoff.** Input: reconciled model and records. Write a short summary of scope and revision, significant risks and uncertainty, selected Day 2 routes and identities, linked T/R/V records, required implementation and test evidence, and the decision owner's role. **Result:** checkpoint 4B artifact. Brief the peer in one minute, separating model-review disposition from checklist status, implementation findings, and a final operational recommendation.

Checkpoints

4A — Feedback is supported, after Step 4. Show a traced flow and a $T \rightarrow R \rightarrow V$ chain. A material comment must name the evidence and requested change. A supported confirmation is valid if no correction is needed.

4B — Reconciled handoff, after Step 7. Every material comment has a disposition. The handoff names concrete implementation paths and evidence rather than saying "run a scanner." Record unresolved matters and owners. Peer agreement alone does not establish that a control is implemented, that the JWAC checklist is satisfied, or that the application is approved.

Day 2 student lab guide — Secure Code Review Practicum

Purpose

Use this guide in delivery order to carry one Field Research Exchange (FRX) review from the reconciled Day 1 handoff through a bounded delegate recommendation. Retain one review document and stable identifiers throughout the day. Do not replace earlier evidence when a later unit adds a status, finding, remediation result, risk judgment, or recommendation.

Record assistance, unresolved evidence, and supported disagreement in the retained review record.

Day 2 sequence

Unit	Learner work	Retained result
1 — Threat-informed review plan	Formal Lab 1	CRP1 revision 0.2 and selected PTH records
2 — Review strategy and Code Crawl	Guided practice	CRP1 revision 0.3 and reproducible crawl handoff
3 — Python source reading	Guided practice	CRP1 revision 0.4 and resumable source trace
4 — Manual baseline crawl	Formal Lab 2	CRP1 revision 0.5, EV evidence, and bounded CF candidates
5 — Risk and governed criteria	Formal Lab 3	CRP1 revision 0.6 and governed RS status records
6 — Signal and supply-chain validation	Formal Lab 4	CRP1 revision 0.7, SV dispositions, and bounded FN records
7 — Remediation and proof	Formal Lab 5	CRP1 revision 0.8, RM dispositions, and remaining-risk evidence
8 — Delegate decision brief	Formal Lab 6	DB1, a 90-second oral brief, and CRP1 revision 0.9

Starting evidence and working files

- Read the [FRX existing-system evidence packet](#) and [risk reference](#).
- Use the [FRX source orientation](#) and the vulnerable learner project under `frx-project/`.
- Maintain the records defined in the **Day 2 record structures** (`labs/day-2-record-templates.docx`).
- Bring the reconciled Day 1 T/R/V/Q and peer-review handoff into CRP1. Do not reconstruct those records from memory.
- The authorized code-review checklist and APPROVE template are controlled, separately supplied inputs. Use them only through the instructor-directed workflow; do not copy their full text into learner work.

Handling notice

The FRX project and all evidence bundles use local synthetic training data and deliberate weaknesses. Keep services on loopback. Do not enter operational credentials or information, connect operational systems, add public targets, or expose the application publicly.

Package setup

From the extracted package root in PowerShell 7.4 or later:

```
./scripts/setup.ps1  
./scripts/verify-environment.ps1
```

Start the local fixture and vulnerable portal only when the instructor directs it. Lab-specific commands and observable checkpoints appear in the relevant section below. Stop and preserve the evidence when integrity, baseline, scope, or authority differs from the guide.

Day 2 Lab 1 — Threat-informed review plan

Assignment and inputs

Produce a review plan that another analyst can execute against the existing FRX learner baseline. Work in pairs. One person traces the Day 1 reasoning and route evidence while the other records; switch after Checkpoint D2-1A.

Record key: A = asset; T = threat scenario; R = security requirement; V = verification; Q = open question; CRP = Code Review Plan; PTH = selected review path; EV = evidence note created in later units.

Use your reconciled Day 1 handoff, the [FRX source orientation](#), the [Day 1 system evidence packet](#), and the **Day 2 record structures** ([labs/day-2-record-templates.docx](#)). You may inspect the source, but this lab does not require running a scanner, proving a vulnerability, assigning checklist status, or changing code.

Create or extend the **FRX review** document. Add a **Code Review Plan** section with CRP1 and a **Selected Paths** section with PTH records. Retain the Day 1 model and T/R/V/Q identifiers you use.

Worked partial plan — CRP1 / PTH1

Retain the Day 1 T1/R1/V1/Q1 chain. Use this first selected path rather than recreating it:

- **PTH1 selection basis:** T1 concerns report data A1 and the unresolved service entitlement decision Q1; R1/V1 require evidence from every report-returning path.
- **Interface and outcome:** `GET /reports/<report_id>` returns the selected report to a signed-in caller or an error.
- **Identity, input, data, effect:** session identity plus caller-selected report ID; report fields return to the browser; a report-read audit event may be written.
- **Source starting points:** `states/vulnerable.py:report_detail`, `_authenticated`, `database.py` report ownership/assignment fields, client `loadReport`, and relevant tests.
- **Proof needed:** locate the trusted-service decision that binds subject to selected report; trace allowed, another valid report ID, and no-session behavior; inspect returned fields and audit/failure effects.
- **Status boundary:** PTH1 is selected for review. It is not yet a confirmed finding and no checklist rule has been marked pass or fail.

Learners extend CRP1 with at least two additional PTH records. Select them because the Day 1 model, consequence, dependency, or coverage need makes the evidence valuable—not simply because the route is short.

Procedure

1. **Establish the decision and baseline.** Input: the Day 1 handoff and source-orientation review target. Record CRP1's decision recipient, purpose, baseline identifier, and whether this is a baseline, change, or hybrid review. State what decision the plan will support and what it cannot decide. **Check:** the plan does not call a Day 1 hypothesis a finding or confuse model disposition with application approval.
2. **Retain the threat-model inputs.** Input: Day 1 scope, model, T/R/V/Q records, residual risk, and peer comments. List the retained IDs and summarize only the evidence, consequence, required behavior,

verification question, and uncertainty needed for code review. **Result:** a traceable input section, not a rewritten threat model. **Check:** another reviewer can find the original record and see what remains unresolved.

3. **Inventory the implementation surface.** Input: the system packet and source-orientation tables. Record the in-scope functionality, identities, assets, stores, external dependency, relevant source families, and existing tests. Name one separately owned dependency or unavailable evidence source. **Check:** include browser/API-only routes, SQLite, file storage, audit output, configuration, and the analytics provider without inventing a report-edit or file-download workflow.
4. **Select high-value paths.** Input: retained Day 1 records and the route-to-source index. Keep PTH1 and add at least two paths with distinct security decisions or effects. For each, record interface, ordinary outcome, identity, input, returned data, effect, source starting points, and selection basis. Complete Checkpoint D2-1A. **Check:** path choice follows an asset, consequence, trust decision, requirement, uncertainty, or necessary coverage—not a generic vulnerability list.
5. **Define the proof needed.** Input: each PTH record and its linked R/V/Q. State the implementation decision to locate, the alternate/bypass/failure path to inspect, and allowed plus denied/failure observations that could support or refute the claim. Identify returned fields, persistent/external effects, and logs to inspect. **Check:** “read the code” and “run a scanner” are activities, not completion conditions.
6. **Allocate support and time.** Input: the source map, authorized references, available tools, and classroom constraints. Assign expertise, reference/checklist access, optional tool support, evidence owner, timebox, and evidence naming convention to each path. Record what remains manual even if a tool reports a match. **Check:** no core outcome depends on internet access, an unavailable enterprise service, or a tool assertion.
7. **Reconcile coverage and stopping rules.** Input: CRP1 and all PTH records. Name deliberate exclusions, separately owned dependencies, open questions, and conditions that require re-planning or escalation. Have your partner explain the path order and proof without your narration; correct missing links or retain the plan with reason. **Result:** CRP1 revision 0.2 ready for Unit 2 strategy review. Complete Checkpoint D2-1B.

Checkpoints

D2-1A — Defensible path selection, after Step 4. Show CRP1 and PTH1 plus at least two learner-selected paths. For each path, point to the retained Day 1 input or coverage reason, ordinary outcome, affected asset/effect, and source starting point. Different path choices are acceptable when the reasoning and evidence are explicit.

D2-1B — Executable plan, after Step 7. A peer can state what to inspect, what proof would support or refute the required behavior, how long to spend, where evidence will be recorded, and when to stop, re-plan, or escalate. The plan visibly separates selected review work from confirmed findings, checklist status, and the final delegate recommendation.

Success means another analyst can begin Unit 4's manual crawl without guessing which baseline, paths, evidence, and completion conditions matter. A long route list without selection reasoning or proof criteria is not a completed plan.

Day 2 Unit 2 guided practice — Strategy and crawl handoff

Purpose and starting material

Refine one selected review path so another reviewer can begin the Python source-reading work in Unit 3. This is guided practice, not a new formal lab. Continue in the retained **FRX review** document rather than creating a separate answer sheet.

Use:

- CRP1 revision 0.2 and its PTH records from Day 2 Lab 1;
- the [FRX source orientation](#);
- the **Day 2 record structures** (`labs/day-2-record-templates.docx`); and
- the reconciled Day 1 T/R/V/Q records linked by the selected path.

Record key: FRX = Field Research Exchange; CRP = Code Review Plan; PTH = selected review path; A = asset; T = threat scenario; R = security requirement; V = verification; Q = open question; EV = evidence note created after later inspection. D2-2A and D2-2B are the Day 2 Unit 2 checkpoints.

Do not create an EV record, finding, checklist pass/fail, not-applicable decision, or final recommendation in this practice. If an authorized checklist copy is available, use only its stable rule IDs in the retained review record; do not copy its controlled text into learner materials.

Worked partial FRX strategy and crawl — PTH1

Retain PTH1 and the T1/R1/V1/Q1 chain. Use this partial answer as the model for the path you refine:

- **Strategy:** bounded baseline review. Day 1 established a reviewed threat model, not a trusted implementation-review baseline. PTH1 includes shared session logic, report storage fields, client use, tests, audit/error behavior, and sibling report-returning paths needed to interpret the report-detail decision.
- **Primary crawl:** selected report ID in the client → report-detail route → current session identity → report selection and subject-object-policy decision → returned fields and audit/error effects.
- **Backward question:** from each returned report field, where did the value, selected record, and release authority originate?
- **Sideways question:** where do shared authentication logic, search, report fields, client rendering, and tests reuse the same assumptions?
- **Branches:** permitted report, another valid report ID, no session, missing report, and audit/error behavior.
- **Evidence plan:** later work must locate the trusted decision and directly observe allowed, alternate-object, no-session, missing-report, returned-field, and audit/error behavior. A plausible alternate explanation is that another trusted layer enforces entitlement.
- **Checklist preparation:** authorization/access control, session, output/data handling, and logging are likely relevant domains. Attach authorized rule IDs and the checklist owner when available. Status remains **Not Reviewed**.

- **Stop or re-plan:** the baseline differs, Q1 policy is unavailable, or a sibling report-returning path changes the coverage claim.

This worked portion selects navigation and evidence questions. It does not disclose a source-level finding or assert that PTH1 passes or fails a rule.

Procedure

1. **Choose one retained path and its strategy.** Starting input: CRP1 revision 0.2 and one PTH record, including PTH1 if desired. Record whether the path uses a baseline, change, or hybrid strategy. Explain the choice using baseline confidence, change reach, affected asset or consequence, and uncertainty. **Check:** the strategy says how far the review must reach; it does not label the path vulnerable.
2. **Name the crawl anchors.** Starting input: the selected PTH and the FRX source orientation. Record the interface or call that begins the path, the identity or authority involved, the asset or material effect, the proof target, and a likely source family. **Check:** every anchor traces to the PTH or retained Day 1 evidence rather than a generic vulnerability list.
3. **Trace three directions and material branches.** Sketch a short forward chain from entry to effect. Add one backward trace from the effect toward its source and authority, and one sideways trace to a shared helper, sibling interface, configuration, dependency, or test. Name at least one alternate, bypass, or failure branch that could change the conclusion. **Result:** a peer-readable crawl route. Complete Checkpoint D2-2A.
4. **Prepare evidence and checklist use.** For the selected path, record the source decision and behavior observations later work must collect, plus a falsifier or alternate explanation. Name likely checklist domains and, if available through the authorized workflow, stable rule IDs and the checklist owner. Record the evidence standard or current authoritative reference to consult. Keep every rule **Not Reviewed**. Add one stop or re-plan condition.
5. **Challenge, revise, and retain the handoff.** Give the path to a partner without narrating it. The partner follows the route and identifies one unsupported jump, missing branch, unclear evidence target, or unresolved owner. Revise the record or retain it with a reason. Update the plan to CRP1 revision 0.3 while preserving the previous revision and PTH ID. Complete Checkpoint D2-2B.

Checkpoints

D2-2A — Reproducible crawl, after Step 3. A peer can explain why the strategy fits, begin at the named anchor, follow the forward chain, and identify the purpose of the backward, sideways, and branch movements. Different routes are acceptable when their evidence links and reasoning are explicit.

D2-2B — Governed Unit 3 handoff, after Step 5. The revised PTH states what Python or Flask construct to locate next, what source and behavior evidence will later support or refute the requirement, which checklist areas may apply, who owns missing evidence, and when to stop or re-plan. It contains no invented EV record, finding, checklist status change, or final recommendation.

Success means another reviewer can open the source orientation and start at the first promised construct without guessing what to follow or why.

Day 2 Unit 3 guided practice — Python source-reading handoff

Purpose and starting material

Turn one retained review path into a source trace that another reviewer can resume during the Unit 4 manual crawl. This is guided practice, not a new formal lab. Continue in the retained **Field Research Exchange (FRX) review** document.

Record key: CRP = Code Review Plan; PTH = selected review path; T = threat scenario; R = security requirement; V = verification; Q = open question; EV = evidence note created from later inspection. D2-3A and D2-3B are the Day 2 Unit 3 checkpoints.

Use:

- CRP1 revision 0.3 and one retained PTH from Unit 2;
- the [FRX source orientation](#);
- the **Day 2 record structures** (`labs/day-2-record-templates.docx`); and
- the reconciled Day 1 T/R/V/Q records linked by the selected path.

Prefer a PTH other than the worked PTH1. If your plan has only PTH1 available, use it and extend the partial example to a different supporting file, branch, or response consumer.

Do not create an EV record, finding, checklist pass/fail or not-applicable decision, or final recommendation in this practice. A source locator shows where to inspect; Unit 4 gathers and interprets line-level and runtime evidence.

Worked partial FRX source trace — PTH1

Retain PTH1 and its T1/R1/V1/Q1 chain. Use this partial answer as the pattern for the different PTH you trace:

- **Registration:** `app.py:create_app` imports the selected state module, calls its configuration hook, and registers its blueprint. The reviewed baseline therefore determines which route implementation is reachable.
- **Entry and input:** `@blueprint.get("/reports/<int:report_id>")` binds the HTTP path to `report_detail(report_id)`. The integer path value and request-session context enter the callable.
- **Identity and decisions:** `_authenticated()` tests session presence. The route then performs a report lookup and handles a missing row. Q1 remains open: where is current subject-object policy evaluated across every report-returning path?
- **Operations and effects:** the route opens the database connection, executes a query, obtains a row, writes an audit message, and returns report fields through `jsonify()`.
- **Sideways support:** `database.py` defines ownership and analyst-assignment fields; `client.js:loadReport` supplies the selected ID and consumes returned fields; `app.py` supplies shared error handling; the tests may state or omit expected permitted, denied-object, no-session, and failure behavior.
- **Next Unit 4 proof question:** which source and runtime observations establish where ownership, assignment, role, or another trusted policy permits or denies release?

This trace records constructs and next questions. It does not assert that PTH1 passes or fails an authorization requirement.

Procedure

1. **Bind the selected entry.** Starting input: CRP1 revision 0.3, one retained PTH, and the route-to-source index. Record the selected state or implementation baseline, application or blueprint registration, route decorator, HTTP method/path, and decorated callable. **Check:** the route is registered in the reviewed baseline; a keyword match alone is insufficient.
2. **Mark inputs, identity, and authority sources.** Starting input: the callable signature, Flask request context, imported helpers, and configuration. Record every material route, query, form, JSON, file, header, cookie/session, or configuration value; its expected type or default; and the next name it receives. Distinguish identity presence from the subject-object or function-authority decision the PTH requires.
3. **Map decisions and branches.** Follow helper results, conditions, early returns, raised exceptions, and error transfers. Record the ordinary path plus relevant missing, denied, malformed, alternate, and failure branches. **Result:** a partner can enter at the same callable and reconstruct the identity and branch map. Complete Checkpoint D2-3A.
4. **Trace exact operations and effects.** Record the fully qualified database, file, serialization, outbound-call, cryptographic, response, audit/log, or exception function used. Name the arguments or values entering it, its return or side effect, and the response or state that follows. Do not infer behavior from a module or keyword when the exact function can be resolved.
5. **Chase bounded supporting code.** Follow only helpers, sibling routes, callers, response consumers, configuration, dependencies, tests, fixtures, and handlers that can change the PTH's input, identity, decision, operation, effect, or interpretation. Record what each supporting source contributes and one missing context or alternate explanation that Unit 4 must test.
6. **Partner-resume and retain the handoff.** Give the trace to a partner without narrating it. The partner reopens the first locator, follows the branch map, and names the next unresolved source construct and proof question. Correct an unsupported jump or retain the trace with a reason. Update the plan to CRP1 revision 0.4 while preserving revision 0.3 and the PTH ID. Complete Checkpoint D2-3B.

Checkpoints

D2-3A — Entry-to-branch trace, after Step 3. A peer can identify how the selected HTTP request reaches the callable, where material inputs and identity state enter, and which branches lead toward ordinary, denied, missing, malformed, or failure outcomes. The trace names unresolved authority rather than assuming it.

D2-3B — Unit 4 manual-crawl handoff, after Step 6. The revised PTH names exact operations and effects, bounded supporting files, the next source locator, and the first line-level or runtime proof question. It remains a source-reading plan: no EV record, finding, checklist status change, or final recommendation has been created.

Success means another reviewer can resume the selected PTH in Unit 4 without asking where the route begins, why each supporting file matters, or what observation is needed next.

Day 2 Lab 2 — Manual Code Crawl-informed trace

Assignment and inputs

Turn retained source traces into reproducible source and runtime evidence against the existing **Field Research Exchange (FRX)** learner baseline. Work in pairs. One person traces source while the other operates the bounded observation and records evidence; switch roles after Checkpoint D2-4A.

Record key: A = asset; T = threat scenario; R = security requirement; V = verification; Q = open question; CRP = Code Review Plan; PTH = selected review path; EV = evidence note; CF = candidate finding.

Use CRP1 revision 0.4, your retained PTH source traces, the [FRX source orientation](#), the [Day 1 system evidence packet](#), the **Day 2 record structures** ([labs/day-2-record-templates.docx](#)), and the supplied **bounded observation runner** ([labs/day-2/lab-2/observe-frx.py](#)).

The runner uses synthetic fixtures and a temporary in-process Flask application. It starts no network listener and does not change the repository runtime. Use only the supplied scenarios and identities. Do not add exploit payloads, public targets, or external services.

From an activated course environment, list the available observations:

```
python labs/day-2/lab-2/observe-frx.py --list
```

Run one observation by name:

```
python labs/day-2/lab-2/observe-frx.py --scenario search-boundary
```

The runner output is evidence input—not a verdict. Interpret it with the reviewed source, intended behavior, retained requirement, effects, and plausible alternate explanations.

Worked partial evidence — PTH1

Retain the Day 1 T1/R1/V1/Q1 chain and Unit 3 PTH1 trace. The following partial answer demonstrates the record standard. Reproduce its runtime observation once, then perform your primary work on another retained PTH.

EV1 — source observation

- **Evidence ID / path:** EV1 / PTH1.
- **Baseline and locator:** learner `vulnerable` state; `states/vulnerable.py:42-45` and `88-99`, functions `_authenticated` and `report_detail`.
- **Direct observation:** `_authenticated` returns an error only when `user_id` is absent from the session. `report_detail` loads the caller-selected integer report ID, returns 404 when no row exists, otherwise records a read event and returns the row. This function does not call an ownership, assignment, role, or other subject-object policy decision.

- **Interpretation:** the traced path establishes session presence but does not show how R1 is enforced for the selected report.
- **Alternate explanation / falsifier:** another registered layer or different active baseline could enforce the release decision before this function. Confirm application assembly and observe the running learner baseline.

EV2 — runtime observation

Run `report-access`. Retain the runner's baseline fields and the observations for no session, report 101, report 102, report 999, and audit output.

- **Direct observation:** after the supplied `partner.alex` login succeeds, `GET /reports/101` and `GET /reports/102` both return status 200 with their respective report IDs and owner IDs; the no-session request returns 401 and the missing report returns 404. Audit output records reads of reports 101 and 102.
- **Interpretation:** the observed report-detail behavior distinguishes authentication, object existence, and the tested cross-owner release. The intended-policy evidence permits this partner to read report 101, not report 102.
- **Limit:** this observation covers one identity, two existing objects, one missing object, and one execution baseline. It does not establish every identity, route, deployment layer, or report state.

CF1 — candidate finding

- **Summary:** the learner baseline's report-detail path permits an authenticated partner to retrieve a report owned by another partner by selecting that report ID.
- **Evidence:** EV1 and EV2; linked T1/R1/V1/Q1 and PTH1.
- **Precondition and effect:** a valid partner session and a known report ID; report fields cross the API response boundary and a successful read is audited.
- **Current disposition:** candidate finding for Unit 5 criteria and governance review. No JWAC checklist rule status, CWE mapping, risk rating, remediation claim, or application recommendation is assigned here.
- **Uncertainty:** confirm whether any deployment-layer control changes reachability and obtain the Q1 policy-owner disposition. Neither uncertainty erases the bounded baseline observation.

Observation choices

Runner scenario	Useful review surface	Evidence to pair with runtime output
<code>search-boundary</code>	PTH2 search input, returned collection, access decision, error branch	<code>search</code> , database query, response fields, client consumer, application 500 handler
<code>document-repeat</code>	PTH4 file identity, persistence, repeated-name behavior, audit	<code>upload_document</code> , configured store, save order, receipt, stored bytes
<code>login-audit</code>	Credential handling, query path, session creation, audit output	<code>login</code> , database lookup, password processing, audit helper, failure branch
<code>error-response</code>	Error body, internal detail, secret/config boundary	<code>error_demo</code> , application handler, configuration, audit behavior

Runner scenario	Useful review surface	Evidence to pair with runtime output
welcome-output	Request value and HTML output context	welcome, response construction, browser interpretation question
analytics-format	Exact parsing/inspection operation and returned processing claim	analytics_import, base64 decoding, pickletools.genops, response fields

Procedure

1. **Bind the observation to the retained plan.** Input: CRP1 revision 0.4 and one retained PTH other than the worked PTH1. Record the active baseline, PTH/T/R/V/Q links, starting locator, required or prohibited behavior, first proof question, evidence owner, and timebox. Run `--list` and select the primary runner scenario that can answer that question. **Check:** the selected observation tests the PTH; it is not an undirected tour of every scenario.
2. **Predict the branches before execution.** Input: the intended behavior, source trace, and ordinary workflow. Build a compact matrix for the primary PTH with at least an allowed or ordinary case, a denied/boundary case, and a missing/failure case. For each, record identity, input/object, expected decision, response/effect to inspect, and what result would challenge the expectation. **Check:** the matrix separates authentication, authorization, validation, existence, processing, and failure when they are distinct decisions.
3. **Capture direct source evidence.** Input: the Unit 3 next locator and branch map. Follow the selected value from request or ambient context through helpers and conditions to its exact database, file, serialization, response, audit/log, or error operation. Create at least one EV record with file/function/line locator, direct observation, linked R/V/Q, and alternate explanation. **Check:** another reviewer can reopen the exact construct and distinguish the code you observed from your interpretation.
4. **Execute bounded paired observations.** Input: the prediction matrix and selected runner scenario. Run the primary scenario, preserve its baseline and containment fields, and record status, response fields, returned object IDs, file hashes, audit entries, or error indicators relevant to each branch. Re-run one observation to confirm determinism. Do not use `--all` as a substitute for planned evidence. **Result:** at least one source EV and one runtime EV for the primary PTH. Complete Checkpoint D2-4A.
5. **Follow effects and failure handling.** Input: the source/runtime EV records. Inspect what leaves the response, what persists, what is overwritten or unchanged, what enters audit output, and which exception or handler changes the result. Add or revise EV records so control, output, side effect, logging, and failure observations remain distinguishable. Select one focused observation from a second PTH and create one additional EV record. **Check:** a 200, 400, 401, 403, 404, or 500 status is evidence only when its identity, input, body/effect, and decision context are recorded.
6. **Reconcile evidence into a bounded disposition.** Input: all EV records and retained R/V/Q. For the primary PTH, choose one disposition: prepare a CF record, record that the tested behavior matched the requirement, or retain an evidence gap. A CF must state the observed behavior, precondition, affected asset/effect, linked evidence, consequence, alternate explanation, and missing owner evidence. **Check:** do not assign checklist status, CWE, final risk, remediation proof, or delegate recommendation. Preserve conflicting source/runtime evidence rather than averaging it away.
7. **Require peer reproduction and retain the handoff.** Input: the primary EV/CF chain. Give your partner only the PTH, source locator, reproduction action, and expected evidence fields. The partner reruns one observation, reopens the source path, and records whether the result matches, differs, or cannot be reproduced. Correct the record or retain the difference with reason. Revise CRP1 to revision 0.5 and

record the completed PTH, focused second-path EV, evidence locations, assistance, gaps, and Unit 5 questions. Complete Checkpoint D2-4B.

Checkpoints

D2-4A — Source-to-runtime chain, after Step 4. Show one primary PTH with a prediction matrix, exact source EV, runtime EV, rerun result, and linked T/R/V/Q. A partner must be able to point from identity/input through the decision and operation to the observed response or side effect. If source and runtime disagree, retain the mismatch and recheck the baseline before proceeding.

D2-4B — Reproducible evidence handoff, after Step 7. Submit CRP1 revision 0.5, the primary PTH with at least two EV records, one focused EV from a second PTH, the peer reproduction result, and zero or more justified CF records. Every CF links to evidence; every no-finding or gap disposition names the tested boundary and remaining uncertainty. Checklist status remains **Not Reviewed** for Unit 5.

Success means another reviewer can reopen the exact source, repeat the bounded observation, see the same output/effect or a retained discrepancy, and understand why a candidate finding, matched behavior, or evidence gap was recorded. Quantity alone does not prove sufficient coverage.

Day 2 Lab 3 — JWAC checklist evidence pass

Outcome and working conditions

Convert retained manual-review evidence into rule-status records another reviewer can challenge. Work in pairs. Keep the evidence author and rule challenger roles distinct; switch after Checkpoint D2-5A.

Record key: T = threat; R = security requirement; V = verification; Q = open question; CRP = Code Review Plan; PTH = selected review path; EV = evidence note; CF = candidate finding; RS = rule-status record.

Use CRP1 revision 0.5, your Lab 2 PTH/EV/CF records, the **Day 2 record structures** (`labs/day-2-record-templates.docx`), the [current-criteria reference](#), and the separately supplied authorized **JWAC Code Review Checklist V1R1, Release 1, benchmark date 15 April 2026**.

The authorized checklist is a governed course input. Use it in the approved environment and do not copy or redistribute the complete benchmark. Work only from the supplied synthetic FRX evidence. Do not add public targets, operational information, or unapproved scans.

What the four statuses mean

Status	Evidence condition
Open	Applicable rule; sufficient evidence demonstrates the required condition is not met
Not A Finding	Applicable rule; sufficient evidence demonstrates the required condition is met for the reviewed scope
Not Applicable	Positive feature/scope evidence demonstrates the rule does not apply; rationale and owner recorded
Not Reviewed	Rule not evaluated, or required evidence has not yet been examined

Local severity helps order review work. It does not decide applicability, status, CWE, final risk, or the application recommendation.

Worked FRX partial answer — RS1

This worked record intentionally completes one part of the lab. Retain it and extend the evidence pass to two additional rules.

- **RS ID / baseline:** RS1; vulnerable learner baseline; Unit 4 review date and peer retained
- **Benchmark:** JWAC Code Review Checklist V1R1; Release 1; 15 April 2026
- **Rule identity:** JWC-CRC-000013 / CRC-AAS-000605; High; access/session domain
- **Applicability:** Applicable to authenticated `GET /reports/<report_id>` access because the path returns a protected report object. The coordinator remains the owner of Q1 entitlement confirmation.
- **Local rule question:** Does the protected report path make and enforce the required subject-object authorization decision before returning the selected record?
- **Current criterion:** OWASP ASVS `v5.0.0-8.2.2`, data-specific access for explicitly permitted consumers

- **Evidence:** PTH1; T1/R1/V1/Q1; EV1 observes session presence and report lookup with no subject-object decision in the function; EV2 observes `partner.alex` receiving owned report 101 and other-owner report 102; CF1 records the bounded difference.
- **Evidence sufficiency:** Sufficient for this path and baseline; a separately registered trusted enforcement layer or different confirmed entitlement policy remains a falsifier.
- **Status: Open**
- **Finding details/comments:** An authenticated partner selected report 102 and received its report/owner fields even though the supplied fixture identifies a different owner. The same path returned 401 without a session and 404 for a missing record, so existence and session decisions do not demonstrate data-specific authorization. Scope is the selected baseline and path.
- **Mapping:** CWE-639 describes the observed user-controlled-key authorization mechanism. OWASP Top 10 `A01:2025` is broader awareness context, not proof.
- **Owner/handoff:** Engineering owns the implementation response; the coordinator owns Q1 policy confirmation. Unit 6 may add or challenge tool evidence. No remediation proof, final risk, or delegate recommendation is assigned.

Learner assignment

Retain RS1. Complete **two additional RS records across at least two other checklist domains**. At least one record must use direct EV evidence; at least one must preserve a local-current conflict, evidence gap, or supported Not Applicable rationale. Use the authorized checklist to retrieve the complete local rule text and metadata.

Suggested identifiers for this bounded exercise:

- build/input: `JWC-CRC-000002` or `JWC-CRC-000005`;
- cryptography/secrets: `JWC-CRC-000017`;
- logging/auditing: `JWC-CRC-000021` or `JWC-CRC-000023`;
- process/supply chain: `JWC-CRC-000025` or `JWC-CRC-000027`.

Steps

1. **Freeze the governed inputs and coverage view.** Input: CRP1 revision 0.5, the authorized checklist, RS1, and all retained EV/CF records. Record benchmark release/date, reviewed baseline, reviewer, the four authorized statuses, and one row for each checklist domain: build/input, access/session, cryptography, logging/auditing, and process/supply chain. Link available evidence or name the gap/owner for each domain. Select the two additional rules. **Check:** selection follows Day 1 consequence, local severity, and available evidence—not the easiest titles.
2. **Decide applicability before status.** Input: each selected rule, its governed domain, implemented features, and review scope. Record Applicable, supported Not Applicable, or unresolved applicability with the feature/scope evidence and owner. Do not use a missing keyword or unexamined artifact as proof of Not Applicable. **Check:** an applicable rule has a specific FRX path, operation, configuration, or process target; a proposed Not Applicable record has positive evidence that the condition is absent from the reviewed scope.
3. **Translate and reconcile the criterion.** Input: authorized local wording and the current-criteria reference. Write one implementation-specific verification question, attach the smallest version-qualified current criterion, and record aligned, local broader, local narrower, ambiguous, or conflict. Preserve

local identity and wording in the governed source. **Result:** two rules with explicit test questions and any conflict/deviation owner.

4. **Link evidence and assign a supported status.** Input: PTH/T/R/V/Q, EV/CF, source/runtime effects, and missing owner evidence. Separate direct observations from interpretation; mark evidence sufficient, conflicting, or gap. Assign Open, Not A Finding, Not Applicable, or Not Reviewed and write finding details/comments that explain the decision. **Check:** another reviewer can move from rule to question to evidence to status without relying on the title. Complete Checkpoint D2-5A.
5. **Add only defensible mappings and decisions.** Input: the observed mechanism and current-criteria reference. Add a precise CWE only when the evidence supports that mechanism; add a version-qualified OWASP category only as context. Record local-current conflict, deviation need, approving owner, and any Unit 6 signal that could change the status. **Check:** severity, Top 10 category, and CWE did not determine status; no unsupported STIG/SRG/control identifier was added.
6. **Require peer challenge and retain the Unit 6 handoff.** Input: RS1 and the two learner-authored RS records. Give the peer the rule ID, test question, evidence links, and proposed status. The peer challenges applicability, sufficiency, mapping, conflict, and owner; record revise, retain with reason, or escalate. Revise CRP1 to revision 0.6 and list every manual, automated, secret, dependency, SBOM, license, threshold, or owner signal Unit 6 must reproduce, reject, or defer. Complete Checkpoint D2-5B.

Checkpoints

D2-5A — Evidence-to-status chain, after Step 4. Show RS1 plus two additional records. Each record preserves benchmark/rule identity, applicability, test question, current criterion, evidence sufficiency, status, and comments. At least one learner record links direct EV evidence; at least one preserves a conflict, gap, or supported Not Applicable rationale.

D2-5B — Governed Unit 6 handoff, after Step 6. Submit CRP1 revision 0.6, a five-domain coverage view, three total RS records including RS1, peer challenge/disposition, deviations or owner questions, and the exact signals Unit 6 must validate. No automated claim is treated as confirmed, no remediation is claimed, and no application recommendation is made.

Completion condition

Your pair is complete when another reviewer can reproduce every evaluated rule's applicability and status from named evidence, understand each current-source clarification or conflict, and identify which unreviewed signals and owners remain for Unit 6. Supported disagreement is retained rather than averaged away.

Lab 4 — Finding and supply-chain validation

Outcome and retained work

Validate a frozen set of manual, static-analysis, secret, dependency, SBOM, and license assertions against the bound Field Research Exchange (FRX) baseline. Retain evidence-backed dispositions that another reviewer can reproduce.

Continue the same review document. Start with CRP1 revision 0.6, the five-domain coverage view, PTH/T/R/V/Q records, Unit 4 EV/CF records, Unit 5 RS records, and the exact signal list from Checkpoint D2-5B. Add:

- **SV** signal-validation records for the worked signal and two learner-selected signals;
- **FN** confirmed-finding records only when direct evidence supports the bounded mechanism;
- updated RS records only when the validated evidence changes their governed status or comments;
- a signal-coverage view and CRP1 revision 0.7; and
- the exact confirmed mechanisms and unresolved evidence Unit 7 may use.

Work in pairs. One learner reproduces; the other challenges baseline, applicability, and disposition. Switch roles at Checkpoint D2-6A.

Handling and decision boundaries

Use only the supplied local, synthetic source and frozen evidence bundle. The integrity helper is read-only, starts no listener, makes no network request, and assigns no disposition. Do not add live scans, public targets, exploit payloads, or real credentials.

The bundle is a transparent course fixture aligned to public report shapes; it is not an authenticated customer pipeline export. Record that limitation. A tool severity, confidence, pipeline result, CVSS value, license action label, or absence of an alert can order work but cannot decide truth, checklist status, risk acceptance, remediation proof, or application approval.

Use these Unit 6 signal dispositions:

- **Confirmed:** the bound assertion and material mechanism are reproduced as stated.
- **Narrowed:** a material mechanism is reproduced, but its scope, mapping, conditions, or consequence is more limited or different than the assertion.
- **Rejected:** direct evidence refutes the assertion for the bound baseline; retain why it looked plausible.
- **Deferred:** the evidence cannot decide; retain the exact gap, owner, and next action.

Starting material

- [Frozen evidence bundle](#)
- **Read-only bundle inspector** (`labs/day-2/lab-4/inspect-evidence-bundle.py`)
- **Day 2 record structures** (`labs/day-2-record-templates.docx`)
- [FRX source orientation](#)
- your retained CRP1 revision 0.6, EV/CF/RS records, and five-domain coverage view

- the separately supplied authorized checklist when an RS record may change

Run the baseline integrity check from this folder:

```
python inspect-evidence-bundle.py --json
```

Expected mechanical result: source and artifact integrity are `true`, the pipeline and baseline are named, signal and component counts are present, `network_used` is `false`, and `dispositions_assigned` is `0`. Stop and notify the instructor if either integrity value is false.

Intentionally shared FRX partial — SV1 and FN1

Retain this worked record; do not repeat it as one of your two learner-selected validations.

- **Signals:** `SAST-001` and `SECRET-001` point to the same bound location in the vulnerable state.
- **Tool assertion:** hard-coded credential/secret; High severity. The SAST report proposes CWE-798.
- **Direct evidence:** the configured `SECRET_KEY` is a fixed string in `configure_app`; Flask uses this key to sign session cookies; the error route also returns the configured value in the bound state.
- **Reconciliation:** two tool alerts support one mechanism, not two findings. The more precise mechanism is a hard-coded cryptographic signing key, CWE-321.
- **Disposition: Confirmed**, with the tool mapping narrowed. Create one `FN1` linked to both signal IDs and the relevant EV/RS chain.
- **Limits:** vulnerable learner baseline only. Exposure in an operational deployment, final risk, remediation, regression proof, and release disposition are not decided here.
- **Next action:** Unit 7 must evaluate a correction and prove session behavior, key handling, and relevant regressions.

Six lab steps

1. **Bind the evidence set and verify scan health.** Input: CRP1 revision 0.6, `pipeline-manifest.json`, `pipeline-policy.json`, and the integrity-check output. Record pipeline ID, application baseline/state, source and artifact integrity, job status, tool/rules/database profile, scan scope and exclusions, SBOM composition, thresholds, and missing customer-specific authority. **Result:** one batch header shared by every SV record. **Check:** another reviewer can tell exactly what was and was not examined before reading a signal.
2. **Separate each assertion from the conclusion.** Input: the retained Unit 5 signal list and every frozen report. Build a short coverage view for manual, SAST, secret, dependency/advisory, SBOM, and license evidence. For each signal, copy its stable ID and exact claim—not its matched secret value—then record locator/component, severity/confidence/score, gate result, likely duplicate, and proof needed. Choose two full validations: one manual/SAST/secret signal other than SV1 and one dependency/SBOM/license signal. **Result:** a triaged queue with two selected claims and explicit reasons.
3. **Reproduce and disposition the first selected claim.** Input: its frozen assertion, bound source, retained EV/CF/RS evidence, and relevant tests or configuration. Follow the claim to the exact operation or value; inspect how the application uses it; look for control, alternate, failure, and contrary evidence; and record one falsifier. Assign Confirmed, Narrowed, Rejected, or Deferred with direct evidence and

owner. Link or de-duplicate against an existing CF/FN/RS rather than counting alerts. **Result:** one complete learner-authored SV record. Complete Checkpoint D2-6A and switch roles.

4. **Validate one supply-chain claim as a chain.** Input: `requirements-lock.txt`, `pyproject.toml`, `frx.cdx.json`, `frx_vulns.json`, `frx_report.txt`, and `pipeline-policy.json`. Check component name, ecosystem/PURL, resolved version, SBOM presence and composition, advisory source/snapshot, affected/fixed range, FRX use or deployment conditions, CVSS and gate threshold, license identifier/expression, missing inventory, and policy owner. **Result:** one complete supply-chain SV record that separates component identity, known-vulnerability match, applicability, threshold action, and license approval.
5. **Reconcile records and governance.** Input: SV1 plus both learner SV records and the Unit 5 RS/coverage view. Merge duplicate alerts that describe one mechanism. Promote only a directly supported mechanism to an FN record; otherwise retain Rejected or Deferred with evidence. Update an RS status or comments only through the authorized checklist workflow. Record threshold/deviation decisions, owners, and why scanner silence does not refute a retained manual CF. **Result:** a signal-coverage view with no double counting and no implicit approval.
6. **Require peer challenge and retain the Unit 7 handoff.** Input: the batch header, three total SV records, any FN/RS updates, and the coverage view. The peer independently checks one locator or component and challenges baseline, scan health, applicability, reproduction, duplicate handling, disposition, threshold, mapping, and owner. Record revise, retain with reason, or escalate. Revise CRP1 to revision 0.7 and list the exact confirmed FN mechanisms Unit 7 may remediate plus every unresolved evidence/deviation owner. Complete Checkpoint D2-6B.

Checkpoints

D2-6A — Reproducible signal disposition, after Step 3. The first learner SV preserves tool assertion, bound baseline, direct evidence, application context, contrary evidence/falsifier, duplicate links, one of the four dispositions, and an owner/next action. A peer can reproduce the evidence without trusting severity or confidence.

D2-6B — Governed Unit 7 handoff, after Step 6. CRP1 revision 0.7 retains earlier revisions; SV1 and two learner SV records cover at least one code/manual/secret and one supply-chain claim; the coverage view includes all six signal families; duplicate alerts are reconciled; FN and RS changes follow direct evidence and authority; threshold/deviation gaps have owners; and Unit 7 receives only confirmed bounded mechanisms. No remediation, final risk revision, or delegate recommendation is claimed.

Completion and reflection

Submit CRP1 revision 0.7, the batch header, SV1, two learner SV records, any linked FN/RS updates, the six-family coverage view, peer disposition, and D2-6A/D2-6B evidence. Record assistance and unresolved evidence.

Be ready to explain:

1. Which evidence changed a tool assertion most substantially?
2. Which passing gate still left review work, and why?
3. Which tool silence could not refute a manual observation?
4. What exact evidence or authority must Unit 7 or the process owner supply next?

Lab 5 — Remediation review

Outcome and retained work

Determine whether a proposed Field Research Exchange (FRX) change corrects a confirmed mechanism without breaking required work. Retain source and behavioral proof that another reviewer can reproduce.

Continue the same review document. Start with CRP1 revision 0.7, the verified Unit 6 batch header, PTH/T/R/V/Q/EV/CF/RS/SV/FN records, the six-family signal view, peer dispositions, and exact proof needs from Checkpoint D2-6B. Add:

- **RM** remediation-review records for the worked correction and one learner-selected confirmed finding/change pair;
- an actionable developer-facing finding statement and four-lane proof matrix for the learner-selected pair;
- any FN or RS evidence/comment update supported through its governing workflow;
- a bounded remaining-risk statement and CRP1 revision 0.8; and
- the exact verified, failed, and unresolved evidence Unit 8 may use.

Work in pairs. One learner reviews the change and executes proof; the other challenges change reach, alternate paths, and over-closure. Switch roles at Checkpoint D2-7A.

Handling and decision boundaries

Use only the supplied local, synthetic change bundle and proof runner. The runner uses temporary Flask test clients, starts no listener, makes no external network request, modifies no repository runtime, reveals no key material, and assigns no remediation disposition. Do not add public targets, real credentials, destructive payloads, or unapproved dependencies.

A source diff, changed line, passing focused check, green pipeline, or absence of the old symptom cannot by itself prove a correction. Directly inspect the root-cause mechanism and test:

- **allowed/ordinary:** required work still succeeds;
- **denied/boundary:** the prohibited or cross-boundary action is blocked;
- **failure/error:** malformed, unavailable, or exceptional behavior fails safely; and
- **regression:** adjacent required behavior and relevant controls still hold.

Use these Unit 7 remediation dispositions:

- **Verified:** the bounded mechanism is corrected and required behavior remains for the tested baseline and paths.
- **Partially Verified:** some proof passes, but a material path, failure, regression, scope, or proof need remains.
- **Not Verified:** the mechanism persists, the correction is absent, or required behavior breaks.
- **Deferred:** integrity, environment, owner, or evidence gaps prevent a decision.

These dispositions describe remediation evidence—not final application risk, residual-risk acceptance, checklist authority, or the Unit 8 delegate recommendation.

Starting material

- [Bounded patch catalog](#)
- **Change manifest** (`labs/day-2/lab-5/change-bundle/change-manifest.json`)
- **Remediation proof runner** (`labs/day-2/lab-5/review-remediation.py`)
- **Day 2 record structures** (`labs/day-2-record-templates.docx`)
- [FRX source orientation](#)
- your retained CRP1 revision 0.7 and exact D2-6B confirmed mechanisms/proof needs
- the separately supplied authorized checklist if an RS record may change

Verify the bound evidence before review:

```
python review-remediation.py --integrity-only
```

Expected mechanical result: source and artifact integrity are `true`; the before baseline, proposed state, five source inputs, and bounded catalog are named. Stop and notify the instructor if either integrity value is false.

List the available focused scenarios when selecting a learner change:

```
python review-remediation.py --list
```

Intentionally shared FRX partial — RM1 for FN1 / PCH-001

Retain this worked record; do not repeat it as the learner-selected remediation review.

- **Confirmed mechanism:** FN1 binds SV1, `SAST-001`, and `SECRET-001` to one fixed Flask session-signing key in the vulnerable source.
- **Correction claim:** PCH-001 removes the fixed assignment, loads a supplied runtime value when present, otherwise generates and retains a separate training-runtime key, and configures Secure, HttpOnly, and SameSite=Lax cookie flags.
- **Change reach:** application configuration, local runtime key file, session-cookie signing, restart continuity, and session-protected routes.
- **Focused proof:** the fixed assignment is absent from the proposed source; the runtime key is created without being returned; it persists across an application restart; a clean runtime gets a different value; ordinary login returns 200; the session cookie includes all three flags; and a report request without a session returns 401.
- **Disposition: Verified for the bounded hard-coded-key mechanism in the remediated training state.** This does not verify production storage or lifecycle.
- **Remaining risk/evidence:** production secret source and access policy, file permission and backup handling, rotation/revocation, previous-session transition, and deployed configuration remain unproved and owned outside this local change.
- **Record effect:** retain FN1 history and add the bounded remediation result. Any governed RS status change requires the authorized checklist workflow. Final risk and the application recommendation remain unassigned.

Reproduce the partial evidence:

```
python review-remediation.py --scenario session-key
```

Six lab steps

1. **Bind a confirmed mechanism to an exact change.** Input: CRP1 revision 0.7, the D2-6B handoff, `change-manifest.json`, integrity output, and the patch catalog. Retain RM1. Select one additional confirmed FN from your or a peer's governed Unit 6 handoff and a matching PCH change. Do not promote a raw, rejected, or deferred signal merely to obtain a patch. Record FN/SV/CF/RS history, before/proposed baselines and hashes, change owner, exact correction claim, and unresolved Unit 6 proof needs. **Result:** one learner RM header with a defensible change-to-finding link. **Check:** another reviewer can identify what changed and why this change is in the queue.
2. **Review root cause and change reach before running proof.** Input: the confirmed mechanism, retained R/V/Q, proposed excerpt, and source orientation. Trace the changed decision, operation, configuration, or value; then inspect callers, shared helpers, identities/objects, data/control flow, side effects, adjacent interfaces, and new assumptions. Write a concise developer-facing finding statement and name one alternate path or new failure mode the change could miss or introduce. **Result:** source proof and a bounded change-impact map—not a line-count summary.
3. **Design four proof lanes and one falsifier.** Input: the correction claim, change-impact map, and Unit 6 proof needs. For allowed/ordinary, denied/boundary, failure/error, and regression, record the precondition, expected result, scenario/observation, and why it tests the mechanism. Include one result that would overturn a Verified disposition. **Result:** an executable proof matrix with no blank lane or assumed outcome. Complete Checkpoint D2-7A and switch roles.
4. **Execute focused proof and preserve observed results.** Input: your selected runner scenario and proof matrix. Run `python review-remediation.py --scenario <name>`. Record source observations, status/effect/audit facts, temporary or mocked dependency limits, and mismatches between expected and observed behavior. Repeat or inspect an adjacent observation when determinism or regression is uncertain. **Result:** direct before/after proof linked to each lane; a passing status without body/effect/identity context is insufficient.
5. **Reconcile the remediation disposition and remaining risk.** Input: the FN/change chain and all proof. Decide Verified, Partially Verified, Not Verified, or Deferred for the exact baseline and scope. Explain whether the root-cause mechanism changed, required work survived, alternate/failure paths were covered, and new risk appeared. Update FN remediation fields; update or propose an RS change only through the authorized checklist workflow. Record remaining risk, confidence, missing operational/deployment evidence, owner, and next verification without accepting that risk. **Result:** one complete learner RM record and actionable developer response.
6. **Require peer challenge and prepare the Unit 8 evidence set.** Input: RM1, the learner RM, CRP1 revision 0.7, linked FN/SV/RS records, and both remaining-risk statements. The peer independently checks one change locator and one proof lane, then challenges finding linkage, mechanism, change reach, falsifier, disposition, checklist effect, remaining risk, and owner. Record revise, retain with reason, or escalate. Revise CRP1 to revision 0.8 and list verified corrections, uncorrected mechanisms, failed/missing proof, governed statuses, deviations, questions, and decision owners. Complete Checkpoint D2-7B. **Result:** a review evidence set for Unit 8—not a recommendation.

Checkpoints

D2-7A — Review-ready proof plan, after Step 3. The learner RM links a confirmed FN to an exact PCH and bound baselines; states the root cause and correction claim; maps change reach and one alternate/new-failure path; and defines allowed, denied, failure, and regression evidence plus a falsifier. No test has been trusted before its purpose and expected observation are stated.

D2-7B — Governed Unit 8 evidence set, after Step 6. CRP1 revision 0.8 retains earlier revisions; RM1 and one learner RM preserve change/source/proof evidence, one of the four dispositions, peer challenge, and bounded remaining risk; FN and RS effects preserve their authorities; verified, failed, and missing evidence have owners. No final application risk revision, residual-risk acceptance, or delegate recommendation is claimed.

Completion and reflection

Submit CRP1 revision 0.8, RM1, one learner-authored RM record, its four-lane proof matrix, any governed FN/RS update, peer disposition, remaining-risk statement, and D2-7A/D2-7B evidence. Record assistance and unresolved evidence.

Be ready to explain:

1. Which proof lane most directly tested the root-cause mechanism?
 2. Which passing observation could have produced false confidence on its own?
 3. What required behavior or adjacent path constrained the correction?
 4. What remaining risk or owner decision must Unit 8 preserve rather than silently accept?
-

Lab 6 — Delegate briefing

Outcome and retained work

Present a concise written and oral recommendation that a Field Research Exchange (FRX) decision owner can evaluate without reconstructing the review. Preserve the evidence, risk, conditions, uncertainty, and authority boundary behind the recommendation.

Continue the same review document. Start with CRP1 revision 0.8, the reconciled five-domain checklist view, six-family signal view, linked T/R/V/Q/PTH/EV/CF/RS/SV/FN/PCH/RM records, verified corrections, uncorrected mechanisms, failed or missing proof, remaining-risk statements, peer dispositions, assumptions, deviations, and owners from Checkpoint D2-7B. Add:

- **DB1**, the delegate-brief record;
- a concise written recommendation and proposed or explicitly unselected customer-template disposition;
- objective conditions, owners, evidence triggers, and consequences when conditions are used;
- a 90-second oral brief and recorded peer challenge; and
- CRP1 revision 0.9 with the decision-owner result left unassigned unless an authorized owner actually responds.

Work in pairs. One learner presents as the review delegate; the other acts as a skeptical decision recipient and checks evidence locators, contrary facts, condition quality, and authority. Switch roles after the first 90-second brief.

Decision and handling boundaries

Use only the supplied course records and the separately supplied customer APPROVE template. Do not copy controlled checklist text, real operational data, credentials, or candidate exam content into the brief.

The delegate makes a bounded recommendation. Only an authorized decision owner decides whether to accept residual risk, approve a deviation or POA&M treatment, sign the organizational record, or issue the actual application disposition. If the authority, required policy, or material evidence is missing, record that gap and recommend Defer and/or Escalate rather than inventing an approval.

Keep two vocabularies separate:

- **Course review action:** Accept, Correct, Reject, Defer, or Escalate.
- **Customer APPROVE recommendation field:** Approve, Approve with conditions, Disapprove, or Disapprove with conditions.

The mapping is reasoned, not automatic. When an APPROVE value cannot be supported, record `Not selected` `-required authority or evidence unresolved`. A POA&M candidate is a governed action record, not remediation proof or risk acceptance.

Starting material

- **Day 2 record structures** (`labs/day-2-record-templates.docx`)

- [FRX existing-system review packet](#)
- [FRX Day 1 risk reference](#)
- **Lab 5 remediation-review guide and worked RM1** (included in this guide)
- your CRP1 revision 0.8 and complete D2-7B evidence set
- the separately supplied customer APPROVE template

Intentionally shared FRX partial — RM1 becomes one DB1 decision line

Retain this decision-input line; do not treat it as the overall FRX recommendation.

- **Reviewed evidence:** RM1 is Verified for the bounded hard-coded session-signing-key mechanism in the remediated training state. The fixed source assignment is absent; temporary-runtime continuity, clean-runtime separation, ordinary login, cookie flags, and no-session denial were observed.
- **Risk effect:** the confirmed source mechanism has changed for the tested state, so it is no longer carried as an uncorrected local mechanism. Do not erase FN1, its vulnerable baseline, or the RM1 scope.
- **Remaining uncertainty:** production secret source/access, permissions/backup, rotation/revocation, previous-session transition, deployed configuration, and decision authority remain unproved.
- **Condition candidate:** the production secret/deployment owner supplies key provenance, access, lifecycle, transition, and deployed-configuration evidence; a named reviewer verifies it before this item can support a production recommendation.
- **Authority boundary:** this line supports the final risk discussion. It does not select the overall review action, an APPROVE value, a POA&M decision, or a signature.

Six lab steps

1. **Bind the decision, recipient, baseline, and authority.** Input: CRP1 revision 0.8, the original service-owner assignment, the reviewed before/proposed states, and the APPROVE template. Record DB1, the exact decision requested, decision recipient, application/build baseline, scope, purpose, operating conditions, timeframe, exclusions, and consequence of delay or incorrect disposition. Name evidence of the authorized decision owner or record `Unresolved-escalation required`. **Result:** a bounded decision header; the learner is not silently cast as the risk-acceptance authority.
2. **Reconcile the evidence set without flattening statuses.** Input: the D2-7B handoff and retained record chain. Build a compact decision view with separate rows for verified corrections, Partially/Not Verified remediation, uncorrected FN mechanisms, governed RS statuses, deferred tool/deployment/policy evidence, failed or missing proof, and rejected claims that must not inflate risk. Link exact record IDs and evidence locators. **Result:** another reviewer can distinguish known fact, supported inference, uncertainty, and inapplicable or rejected material.
3. **Update the material risk arguments.** Input: the Day 1 risk records, supplied APPROVE matrix, current evidence view, operational requirements, and remaining-risk statements. For each decision-driving line, state threat event, supported weakness or predisposing condition, mitigating evidence, security references, likelihood, impact, risk result, confidence, assumptions/uncertainty, and the evidence that could move the result. Do not average unrelated findings or let checklist severity/tool confidence decide risk. **Result:** a small set of traceable risk lines. Complete Checkpoint D2-8A.

4. **Select and translate the recommendation.** Input: the decision header and risk lines. Choose Accept, Correct, Reject, Defer, or Escalate for the exact requested scope and justify it with the strongest supporting and contrary evidence. Separately propose one of the four APPROVE template values only if the evidence and assignment support it; otherwise state why the field remains unselected. **Result:** one bounded recommendation, not a menu of unlabeled options.
5. **Write the executive summary and objective conditions.** Input: DB1 and the template fields. State the decision requested, baseline/scope, most material supported risk, verified mitigation, remaining risk, recommendation, and authority boundary. For each condition, name owner, required action, observable completion evidence, due date or event trigger, re-review owner, and consequence if unmet. Record non-compliance, assumptions, constraints, operating limits, and releasability/FGI as evidence or named gaps. Link POA&M/deviation treatment only when its authority is established. **Result:** a concise written brief a decision owner can act on.
6. **Deliver, challenge, and preserve the actual decision boundary.** Input: the written brief and source records. Deliver a 90-second oral brief in this order: decision requested; scope/baseline; strongest evidence; most material remaining risk; recommendation/conditions; authority and next action. The peer checks one decisive locator, states the strongest contrary fact or alternate recommendation, and challenges scope, risk, conditions, and authority. Record revise, retain with reason, or escalate. Revise CRP1 to revision 0.9 and complete Checkpoint D2-8B. **Result:** DB1 and an oral disposition that remain distinguishable from any later authorized decision.

Checkpoints

D2-8A — Decision-ready evidence and risk, after Step 3. DB1 binds the decision, recipient, baseline, scope, timeframe, and named or unresolved authority. The evidence view preserves distinct RS/SV/FN/RM meanings; each material risk line links the threat event, supported mechanism or condition, mitigation, likelihood, impact, risk, confidence, uncertainty, and evidence that could change it. Rejected claims and missing evidence are not converted into findings.

D2-8B — Challenge-ready delegate brief, after Step 6. CRP1 revision 0.9 retains earlier records; DB1 states one bounded course review action, a separately justified or explicitly unselected APPROVE field, objective conditions, releasability/FGI evidence or gap, peer challenge, and a 90-second oral brief. The record names who may decide and leaves the actual signature, risk acceptance, deviation/POA&M approval, and organizational disposition unassigned unless an authorized owner responded.

Submit

Submit CRP1 revision 0.9, DB1, the compact evidence/risk view, written executive summary, recommendation and template-field rationale, conditions or explicit reason none apply, peer challenge/disposition, and your 90-second oral brief outline. Record assistance and unresolved evidence.

Field Research Exchange — existing-system review packet

Review baseline: **FRX-LEARNER-BASELINE**, the implementation described by this packet and its supplied diagrams. Use that label in your review scope; it identifies this course evidence baseline, not a deployed production release.

Read the assignment

Field Research Exchange (FRX) lets a research operations team read partner field observations before its weekly planning meeting. A **report** is a registered observation with a title, summary, owner, classification label, and optional assigned analyst. An **intake document** is a submitted file awaiting handling outside the portal. An **analytics score** is advisory information used to choose which report to read next.

The service owner has requested a baseline security review of the existing application. Your team will model its current behavior, identify plausible threats, recommend responses with verification criteria, and subject that work to peer review. Day 1 produces a defensible model and implementation-review questions for Day 2. The service owner retains the operational decision; completing a model does not certify the application or confer certification on the reviewer.

Handling notice: use only the supplied local course application, accounts, and data. Do not enter operational information or contact external targets.

Evidence status and owners

The stakeholder statements below are authored case facts, not customer policy. Engineering descriptions reflect the supplied baseline application. A successful normal request demonstrates that path; it does not establish that unauthorized requests are rejected.

ID	Source	Evidence supplied
E1	Service owner — review brief	Review the current portal and client. Unauthorized disclosure can expose a partner's observations; changed or misattributed records can distort planning. Record supported risks and missing evidence before recommending disposition.
E2	Research coordinator — access and workflow requirements	Partners may read their own reports; analysts may read assigned reports; administrators may troubleshoot across reports. Apply this intent to search results and report-related analytics as well as direct report reads. The coordinator registers reports and assigns analysts through managed intake outside this application.

ID	Source	Evidence supplied
E3	Engineering — supplied system documentation	The architecture, sequences, route inventory, and data model below describe the implemented baseline. No separate gateway, identity provider, queue, or deployed network segmentation is evidenced.
E4	Operations — operating requirements	Normal review work is weekdays 08:00–18:00. An interruption up to two hours is tolerable with notice to the coordinator. Acknowledged intake documents must not be lost. Reports must remain readable when analytics is unavailable. These are requirements; restore capability and operating procedures have not been demonstrated.
E5	Analyst — use of analytics	A score helps order work; the report remains the source for the observation. The analyst needs to know which report a score describes and must be able to continue reading without requesting a score.
E6	Engineering — normal demonstration	Sign in, search for Prairie, open report 101, request its score, submit a text document, and sign out. The expected observations appear below and in the supplied sequences.

People and records

Account	Role	Intended access from E2
partner.alex (user 1)	Partner	Report 101
partner.jordan (user 2)	Partner	Reports 102 and 103
analyst.riley (user 3)	Analyst	Reports 101 and 103
admin.casey (user 4)	Administrator	All three reports

Report	Title	Owner	Assigned analyst
101	Prairie Sensor Review	1	3
102	Watershed Sampling Review	2	None
103	Remote Station Review	2	3

All three records carry the application's `partner-observation` label. It is a case data label, not a government classification marking. Ownership and assignment are data used by the intended policy; their presence does not prove enforcement.

Follow the ordinary workflow

1. Open the supplied browser client and sign in with the account supplied by the instructor. The client displays the username and role and automatically requests a report list using an empty search.
2. Enter **Prairie** in search. The portal queries report titles and summaries and returns report 101. The list includes the report summary and owner ID, so search itself is a disclosure surface.
3. Open **Prairie Sensor Review**. The detail panel displays report 101, owner 1, analyst 3, the classification label, and the summary about sensor calibration drift. A selected report number is an input to the service, not proof of permission to read it.
4. Request its analytics score. The client calls the portal, which calls the analytics provider. The supplied provider returns report 101 with score **7**. The client displays the score; it does not save it into the report.
5. Submit a short text document through document intake. The receipt contains a stored filename. Selecting report 101 beforehand does not associate that file with report 101. There is no file retrieval screen or endpoint in this build.
6. Sign out. The portal clears the session and the client returns to sign-in.

These normal observations can be traced without using an attack input. Use the [diagram collection](#) to follow the messages, or the individual images in the [diagram index](#).

Interfaces and effects

The table describes the baseline reviewed on Day 1. Login requirements describe the normal route contract; their completeness and effectiveness remain review questions. API-only entries are part of the scope even though the sample client does not expose a button for them.

Interface	Input and ordinary result	Persistent effect / client use
GET / and /static/<path>	Browser page, script, stylesheet	None; client loads these resources
GET /health	Service status and state	None; client checks availability
POST /login	Username/password → user ID, name, role and session cookie	Login events; client sign-in
POST /logout	Session → logged-out response	Logout event and cookie/session clearing; client sign-out
GET /welcome?name=...	Name → HTML greeting	None; public API-only route
GET /search?q=...	Signed-in search text → matching report records	Database read; no search audit event; client search/list
GET /reports/<report_id>	Signed-in report selection → record or error	Database read; successful read event; client detail
POST /documents	Signed-in multipart field <code>document</code> → filename receipt	File write and upload event; client intake
POST /analytics/import	Signed-in submitted analytics payload → processing result	No database/file persistence; API-only input, not the upload form
GET /analytics/fetch?url=...	Signed-in provider URL → provider response	Outbound request; no saved score; client score button

Interface	Input and ordinary result	Persistent effect / client use
<code>GET /errors/demo</code>	Signed-in diagnostic request → error response	No new application record; API-only diagnostic surface

The provider exposes `GET /health` and `GET /metrics/<report_id>`. Its metrics response includes report ID, score, category, and source. It is a separate service and has no direct connection to the portal database. The baseline analytics import accepts a base64-encoded serialized payload and returns inspection metadata; it is not an implemented report update. The later reference build has different request contracts and must not silently replace this baseline documentation.

Engineering context for import (authored case fact): this API-only interface is retained for inspecting received analytics exports. It is outside the ordinary planning workflow but inside the review scope because it accepts signed-in callers' data. The supplied implementation demonstrates inspection, not integration of an export into a report or an analyst approval workflow.

Data and system responsibilities

The browser runs the sample client, retains a session cookie, and displays reports and receipts. Flask serves the client and handles API requests. SQLite stores users (including password-verification data and role) and reports. The portal writes incoming files to a separate intake directory and selected events to an audit file. Those three persistent stores are separate; a successful file upload is not a database transaction or proof of recoverability.

Managed intake is an external business process. Its outputs are the registered records and assignments represented by the initial database contents. There is no live registration connection implemented in the portal. Use the context view to understand that business dependency; do not infer an intake API from its dotted relationship.

Review the browser/client, portal routes, database interactions, file handling, event/error output, and the portal's provider interaction. Treat the managed-intake process and provider internals as external dependencies whose assumptions matter. The available implementation is local topology, not evidence of a production deployment design.

Questions with evidence owners

ID	Question to investigate	Owner / useful evidence
Q1	Where and how is E2 enforced for each report-returning path?	Engineering: route review and allowed/denied request evidence
Q2	Who may access intake files after acknowledgement, and how are loss and duplicates handled?	Coordinator and operations: intake procedure, filesystem permissions, restore demonstration
Q3	How does the portal establish the intended provider destination and the identity of the returned report?	Engineering/provider owner: request construction, response validation, dependency contract
Q4	Which events are needed to explain a disputed read or upload, and who can read or alter the logs?	Service owner/operations: audit requirements, access/retention settings, sample events

ID	Question to investigate	Owner / useful evidence
Q5	What protects traffic, application identity, and stored data in the intended operating environment?	Operations: proposed deployment, key/access management and backup evidence; none supplied as deployed fact

You have enough architecture to begin modeling. Carry consequential unanswered questions forward explicitly rather than inventing controls. Use E1–E6, route names, and the model's stable IDs to retain the source of each judgment.

From system documentation to a threat-model diagram

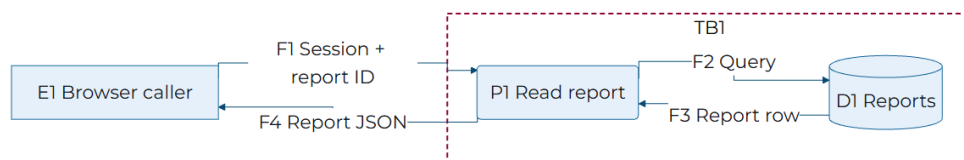
An architecture view tells you what is connected. A sequence follows one interaction in time. A threat-model DFD makes data movement and changes in trust explicit so you can ask what can go wrong. Start from the supplied system evidence; use this course legend consistently. Other recognizable notations are acceptable if you provide a legend.

Mark	Meaning	FRX example
Rectangle, E ID	External entity relative to the modeled process	Browser caller supplying a report selection
Rounded box, P ID	Process that receives, transforms, or releases data	Read report
Cylinder, D ID	Persistent data store; course convention	Report records in SQLite
Directed arrow, F ID	Named data movement; arrow points toward recipient	Session cookie and report ID
Dashed enclosure, TB ID	Boundary justified by a change in authority or trust	Caller-controlled request entering portal authority

Shapes and line patterns carry meaning without relying on color. A boundary says why validation or authorization matters; it does not assert an effective control. Keep architecture grouping separate from boundary claims. A filesystem store is not a service that authorizes its own requests.

Worked partial answer: report read

Worked partial answer — report-read DFD



Legend: rectangle = external entity; rounded box = process; cylinder = store; directed arrow = data flow; dashed enclosure = justified trust boundary. Partial answer only; see the notation guide.

- Read the evidence.** E2 limits partners to their own reports. E6 and the report-read sequence show the browser selecting 101, the portal reading SQLite, and the response returning report fields.
- Select the level.** Model the report-read operation as P1. E1 is the browser caller relative to that server operation, although the browser client itself remains in the overall review scope. D1 represents the report data within SQLite; the database also holds users, omitted from this fragment.
- Name the flows.** F1 carries the session cookie and report ID into P1. F2 is the lookup request, F3 the returned fields, and F4 the JSON delivered to the caller. Separate directions make both supplied input and released data visible.
- Explain the boundary.** TB1 separates caller-controlled selection from the portal's authority to retrieve records. Being signed in identifies a session; it does not prove entitlement to the selected record. The

drawn enclosure does not claim a separate database security zone.

5. **Connect an asset.** A1 = report observations and ownership/assignment metadata, illustrated here by report 101. A1 covers this data category across the collection, not only Alex's record. Business owner = research coordinator, with each partner as originator. Protection need = restrict disclosure according to E2 and preserve accurate attribution so planning is not based on altered observations (evidence E1).
6. **Retain uncertainty.** Q1 asks how access is enforced across report-returning paths. A normal successful read cannot answer it. Later scenario work can hypothesize an unauthorized read; label that hypothesis until evidence establishes the failure.

This is an intentionally shared partial answer. It omits login, search, upload, audit output, diagnostics, analytics, and further boundaries. Extend it from the supplied architecture and interface inventory in Lab 1. Retain these IDs where you reuse these elements; assign new IDs to added elements. Evidence E1–E6 and entity IDs are separate namespaces: write “evidence E1” or “entity E1” when ambiguous.

Context, detail, and a readability check

A context model treats FRX as one process and shows its external exchanges. A detailed model decomposes that process into relevant operations and stores. Preserve the meaning of incoming and outgoing exchanges when decomposing; the supplied business context's dotted managed-intake relationship is not a live HTTP flow. Do not draw an implemented registration endpoint to fill that gap.

Trace a request with a finger: where does its data enter, who acts on it, what is read or written, and what leaves? Every arrow needs a meaningful payload and direction. Check that each added boundary has a stated reason and each unknown has an evidence owner. A peer should be able to trace upload separately from report read without assuming the file is part of the report.

Mermaid is used to author these supplied diagrams. You may use paper or an ordinary editor; the model's meaning and traceability are the work products.

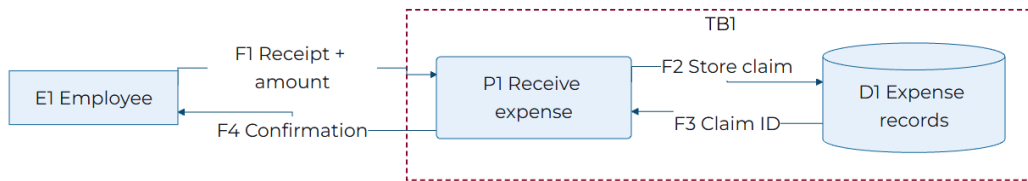
Reference basis: [OWASP Threat Modeling Cheat Sheet, System Modeling](#), accessed 8 September 2026. It supports documenting processes, stores, entities, flows and trust boundaries, with overview/detail views; the shapes, IDs, and FRX example above are course conventions.

Supplied FRX diagram collection

These diagrams are views of the system and threat model. Validate each view against the evidence packet and source; the model is the maintained body of scope, evidence, assumptions, scenarios, requirements, and decisions, not any single picture.

Generic DFD example

Generic example — expense-submission DFD

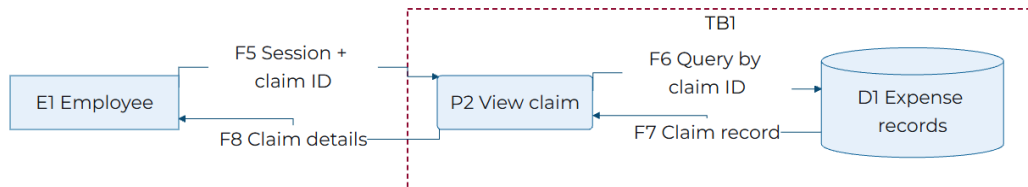


Course-convention example. The actor is outside the modeled application, arrows name data and direction, the process acts on data, the store persists it, and the dashed enclosure marks a justified authority change.

Actor, process, store, flow, and trust-boundary notation before FRX.

Generic threat-analysis DFD

Generic example — expense-record lookup DFD

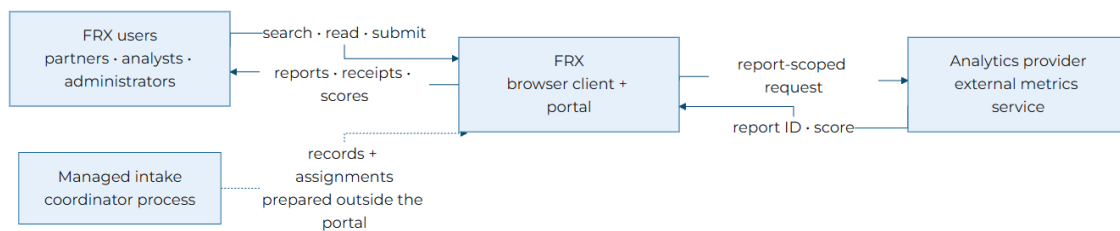


Threat-analysis example. The DFD establishes a caller-controlled claim ID, server process, record query, stored claim, and returned details. It does not establish SQL construction or object-level authorization behavior.

The same system annotated with separate injection and object-authorization hypotheses.

FRX system context

FRX system context

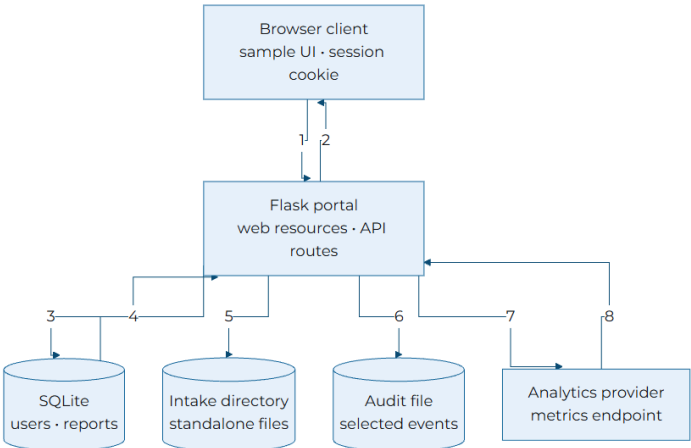


Business context. Dotted relationship is external preparation of records, not a live portal API.

People, the portal, managed intake, and analytics-provider relationships.

FRX implementation architecture

FRX implementation architecture

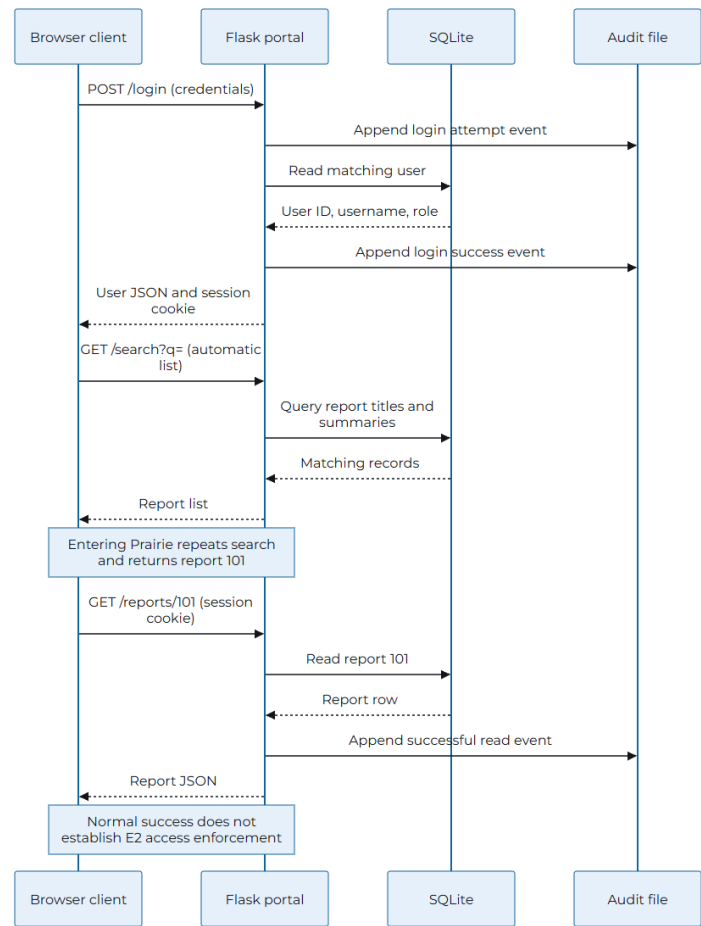


Architecture flowchart of the supplied local implementation. 1 requests and input; 2 HTML, JSON and cookie headers; 3 queries; 4 rows; 5 file bytes; 6 selected events; 7 metrics request; 8 JSON result. Connections do not prove security controls. Managed intake is external and has no live connection here.

Browser/client, Flask routes, SQLite, intake files, audit output, and analytics provider.

FRX report-read sequence

Sign in, search, and read a report

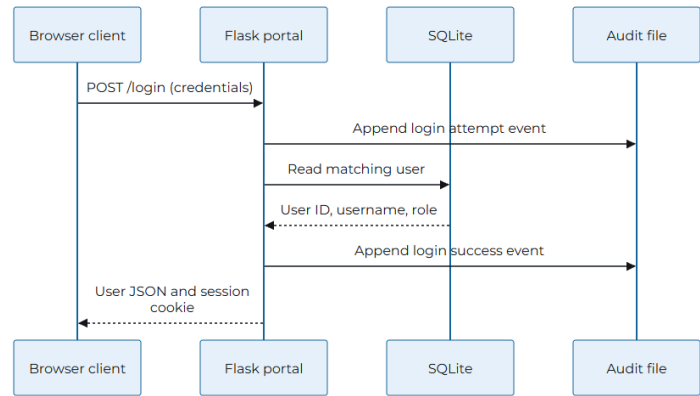


UML-style normal-success sequence for the baseline. Session cookies accompany search and report requests. Search has no audit event. E2 access requirements remain to be verified.

Sign-in, automatic search, report selection, and returned data.

FRX sign-in detail

Report workflow — sign in

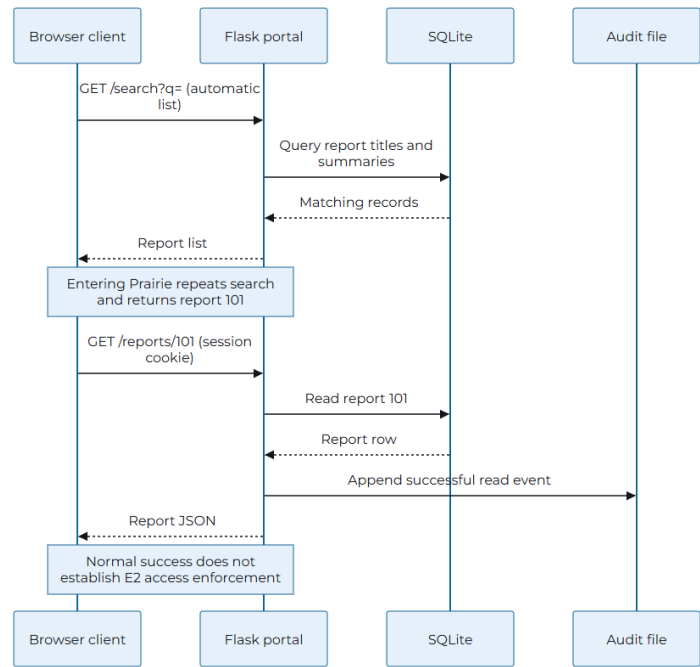


First part of the report-read sequence. A normal successful sign-in establishes a session; it does not demonstrate report entitlement.

The identity-establishment portion of the report-read path.

FRX search/read detail

Report workflow — search and read

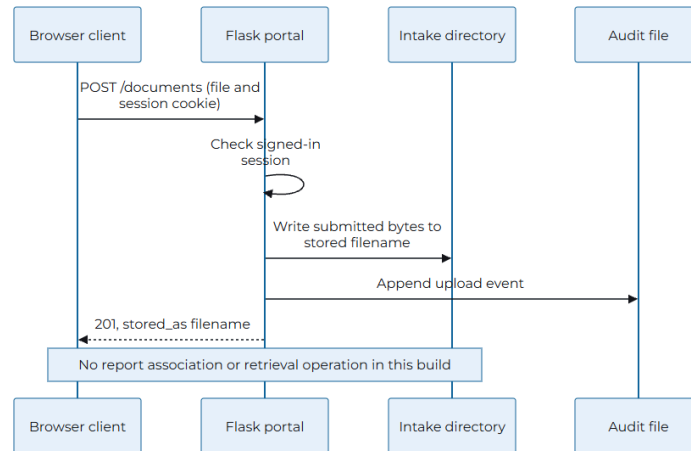


Second part of the report-read sequence. Search and read include the session cookie. E2 access requirements remain to be verified.

The report-selection and report-return portion of the same path.

FRX document-intake sequence

Submit a standalone document

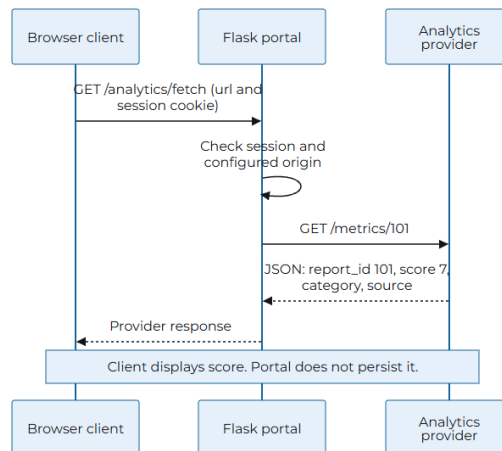


Normal successful baseline upload. The receipt acknowledges file storage, not report association or recoverability.

File acknowledgement and storage, independent of the report database.

FRX analytics-fetch sequence

Request an analytics score

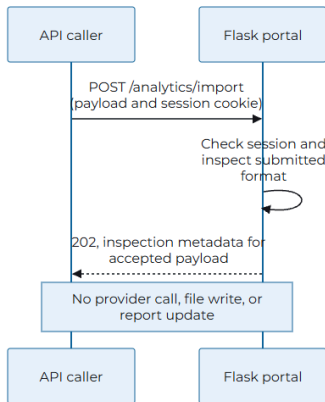


Baseline request contract. Provider JSON includes report identity; a normal response does not establish validation for other responses.

Portal-to-provider request and advisory score response.

FRX analytics-import sequence

Inspect submitted analytics data

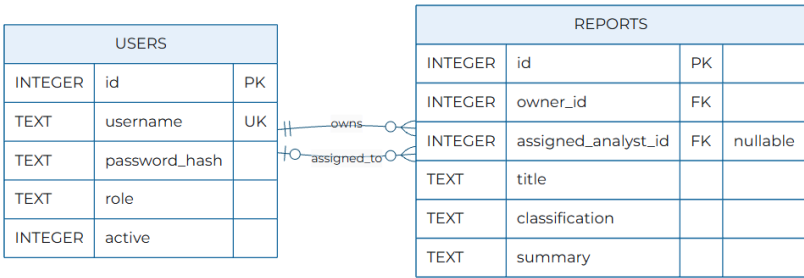


Baseline API-only path. Accepted input is a base64-encoded serialized payload; returned metadata describes inspection without execution or persistence.

The API-only analytics representation-processing path.

FRX data model

Users and reports — relational data model

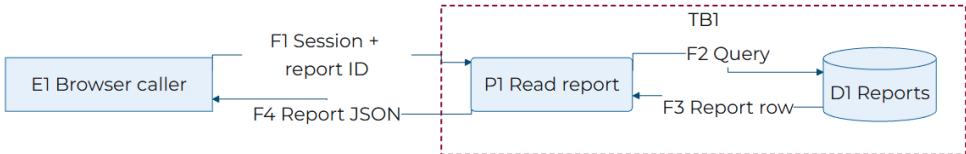


Declared schema relationships: each report has one owner and zero or one assigned analyst. Role semantics and enforcement are separate from foreign-key declarations. Files, logs, and the browser cookie are not tables.

Declared user and report fields; schema presence is not authorization proof.

FRX report-read model fragment

Worked partial answer — report-read DFD



Legend: rectangle = external entity; rounded box = process; cylinder = store; directed arrow = data flow; dashed enclosure = justified trust boundary. Partial answer only; see the notation guide.

The intentionally partial DFD fragment used in the notation lesson.

Threat discovery reference — STRIDE, OWASP and CWE

Use this reference with your FRX model; no live web research is required during the lab. Summaries below are paraphrases. Sources were checked on 8 September 2026. These prompts do not establish that FRX contains a vulnerability.

Read a DFD as an investigation map

A DFD establishes actors, processes, stores, flows, assets, and changes in authority. Those features identify useful questions; they do not prove a weakness.

- **Database path:** ask whether externally influenced data reaches SQL construction and could be interpreted as SQL rather than ordinary data. A database alone does not establish SQL injection. Parameter binding or another structure-preserving query mechanism may disprove that hypothesis.
- **Object or asset path:** ask whether a caller-controlled record reference selects protected data and whether the trusted service enforces the caller's entitlement to that particular object. An asset or sequential identifier alone does not establish IDOR.
- **Boundary-crossing path:** ask what authenticates the actor, authorizes the requested object or action, preserves data meaning, constrains resource use, and records the event. A boundary marks a change in control or authority; it does not prove that those controls exist or work.

Use the model to select a path, apply all six STRIDE questions, use the OWASP Top 10 to broaden plausible mechanisms, and choose a CWE only after the mechanism is specific enough to support it.

Apply STRIDE to a specific element or flow

Category	Question to ask	FRX investigation surface
Spoofing	Could an actor be accepted as a different identity?	Sign-in, session cookie, provider identity
Tampering	Could someone alter data without authority?	Intake bytes/name, query input, provider result
Repudiation	Could a disputed action lack reliable attribution?	Read/upload events and their protection
Information disclosure	Could data reach someone not entitled to it?	Report/search output, logs, diagnostics, browser rendering
Denial of service	Could legitimate work become unavailable?	Request volume, file storage, provider delay
Elevation of privilege	Could a lower-privilege actor gain greater authority?	Role/session handling and privileged data access

Apply all six as questions, then retain supported scenarios or explain why a prompt does not fit. You need not invent one scenario per category for every node. One scenario may involve several categories; choose a

primary category that explains the consequence. [Microsoft STRIDE descriptions](#), updated 25 August 2022, supply the category basis; the FRX prompts are authored examples.

Use OWASP Top 10:2025 to broaden the mechanism

Category	Retrieval/application prompt
A01 Broken Access Control	Which resource may this caller act on?
A02 Security Misconfiguration	What deployment/configuration evidence is missing?
A03 Software Supply Chain Failures	Which dependency/provenance evidence needs later review?
A04 Cryptographic Failures	What protects sensitive bytes and keys?
A05 Injection	Where could data become query, markup or other instructions?
A06 Insecure Design	Does the workflow enforce its business invariant?
A07 Authentication Failures	What establishes and maintains caller identity?
A08 Software or Data Integrity Failures	What establishes trust in received data or software?
A09 Security Logging and Alerting Failures	Could a material event be missed or misinterpreted?
A10 Mishandling of Exceptional Conditions	What happens when input, storage or a dependency fails?

Source: [OWASP Top 10:2025 introduction](#). This awareness list is neither a full verification checklist nor a STRIDE-to-CWE lookup table. A provider identity scenario might use A08; an unauthorized destination scenario might use A01. Explain the mechanism rather than mapping every external-service issue to one category.

Choose a CWE only when its mechanism fits

MITRE CWE version 4.20 entries, accessed 8 September 2026:

Entry	Use when your scenario specifically hypothesizes...
CWE-89 — Improper Neutralization of Special Elements used in an SQL Command	Externally influenced input is used to construct SQL and can alter the intended command instead of remaining ordinary data
CWE-639 — Authorization Bypass Through User-Controlled Key	A caller changes a record key and gains unauthorized access to someone else's data
CWE-862 — Missing Authorization	A required authorization check is absent; do not assert absence without evidence
CWE-79 — Improper Neutralization of Input During Web Page Generation	Untrusted content reaches a web page in an executable context without appropriate neutralization
CWE-73 — External Control of File Name or Path	Caller influence over a storage name/path enables an unauthorized file effect
CWE-117 — Improper Output Neutralization for Logs	Data entering a log can be interpreted as misleading record structure

Entry	Use when your scenario specifically hypothesizes...
CWE-918 — Server-Side Request Forgery	Caller influence induces a server request to an unintended destination/resource
CWE-770 — Allocation of Resources Without Limits or Throttling	Unbounded allocation or request frequency exhausts resources

IDOR is an access-control vulnerability in which an exposed object reference is changed and the application fails to verify authorization for the referenced object. CWE-639 is appropriate when that caller-controlled-key mechanism is the hypothesis; CWE-862 is broader and applies when an authorization check is absent. Do not use either merely because a protected asset or numeric ID exists.

Do not claim arbitrary network access from FRX: its provider origin is constrained in the supplied build. Do not claim executable deserialization from an interface documented as inspection only. A CWE describes a weakness mechanism; a threat scenario also needs actor/event, precondition, path, asset and consequence. If the mechanism is uncertain, write “CWE unresolved pending ...” and identify the needed evidence. That is better than a precise-looking unsupported mapping.

PASTA in context

PASTA means Process for Attack Simulation and Threat Analysis. Its seven stages move through objectives, technical scope, application decomposition, threat analysis, weakness/vulnerability analysis, attack modeling, and risk/impact analysis. It provides a broader risk-centered process in which discovery techniques may be used. STRIDE provides categories for asking threat questions; it does not replace scoping, risk decisions or validation.

For FRX, E1 supplies an objective, the system packet supports scope/decomposition, and Q1 identifies evidence needed before treating an attack path as established. We practice the four-lab STRIDE-led workflow, not a complete PASTA engagement. Source: [VerSprite, Benefits of PASTA Threat Modeling](#), methodology-author organization, living page accessed 8 September 2026.

Risk and Response reference

Build a risk argument before selecting a response

Retain the Lab 2 threat scenario unchanged. Add a risk judgment that explains:

1. **Likelihood:** how the complete event could occur, using its access, preconditions, path, supported controls, and unresolved control behavior.
2. **Impact:** what harm follows if the event occurs, using the affected asset, stakeholder consequence, breadth, duration, and recoverability.
3. **Risk result and confidence:** the supplied matrix cell, how strongly the evidence supports it, and which assumption or observation could move it.
4. **Relative priority:** why this record should be addressed before, after, or alongside another record—including urgency created by consequential uncertainty.

A reachable interface is evidence for the path, not proof that a hypothesized failure is repeatable. An unresolved control is uncertainty, not evidence that the control is absent. Name the evidence source and owner that could change the judgment. NIST SP 800-30 Rev. 1 describes risk as a combination of likelihood and impact and calls for explicit treatment of uncertainty; the workshop scales below are a simpler authored practice convention.

Workshop reporting matrix

This matrix is an authored practice convention, not a JWAC policy, STIG severity mapping or certification threshold. Use the organization's approved criteria in operational work. Assess the scenario in the intended use described by E1–E5; local classroom containment is a handling control, not proof of production likelihood.

Likelihood	Evidence anchor
Low	Preconditions are restrictive and supported controls materially impede the path; identify that evidence
Medium	A credible reachable path exists, but a consequential control or prerequisite remains unresolved
High	Evidence supports a readily repeatable path with little additional access or effort; identify the evidence, not just its theoretical possibility

Impact	Case consequence anchor
Low	Limited, recoverable inconvenience without unauthorized report disclosure or loss of acknowledged work
Medium	Material delay/rework or a bounded loss of confidence requiring coordinator intervention, with recovery plausible

Impact	Case consequence anchor
High	Unauthorized partner-report disclosure, altered observations/attribution affecting decisions, unrecoverable acknowledged work, or sustained loss of required review capability

Likelihood ↓ / Impact →	Low	Medium	High
Low	Low	Low	Medium
Medium	Low	Medium	High
High	Medium	High	High

Record confidence and the assumptions that could move a rating. The matrix groups priorities; break ties using evidence urgency, breadth, dependency and consequence. Do not invent numeric precision or treat an unknown control as proven absent.

Responses and verifiable requirements

NIST risk guidance distinguishes accepting, avoiding, mitigating, sharing, and transferring risk. For this lab, use **mitigate**, **avoid**, **share/transfer**, or recommend **acceptance** while stating any retained responsibility. Compare plausible alternatives, legitimate-work consequences, interim measures, accountable owners, effectiveness criteria, and reassessment triggers. A reviewer recommends; the service owner decides. “Ask someone else” is an evidence action, not itself a completed response.

A requirement names the actor/resource, trigger, required behavior, disallowed behavior, relevant failure condition, basis, and owner. State observable behavior before naming a library or implementation. A verification record names setup, action, allowed observation, denied or failure observation, and what would falsify the claim. Inspect response content and persistent effects as well as status. Include allowed cases so a mitigation cannot pass by breaking legitimate work.

For report access, [OWASP ASVS v5.0.0, V8](#) supports documenting function- and data-specific rules (8.1.1), restricting access to specific data items (8.2.2), and enforcing decisions in a trusted service layer rather than relying on a manipulable consumer (8.3.1). These are paraphrases; use version-qualified identifiers. Our R/V IDs are local work-product IDs, not invented control identifiers. No checklist or exam disposition is implied by a Day 1 risk rating.

After proposing a response, record residual risk, missing evidence, responsible owner, next action, and the fact or change that should reopen the judgment. Proposed response, implemented control, and verified behavior are three different states.

DREAD: recognize the method and its limits

DREAD considers Damage potential, Reproducibility, Exploitability, Affected users and Discoverability. One historical convention assigns each 1–10 and averages the five. Illustrative T1 assumptions might yield 8, 6, 7, 5, 8: mean 6.8. Here damage assumes unauthorized disclosure; repeatability and exploitability assume the check is absent; affected users reflects an assumed scope; discoverability assumes selectable IDs. None of those ratings is measured FRX evidence.

If repeatability changes from 6 to 2 after finding an effective control, the mean becomes 6.0. The apparent precision conceals changed assumptions and possible double-counting. Discuss that limitation; use the supplied likelihood/impact matrix for the lab. Historical source: [Microsoft, Security Briefs, March 2010](#), explaining the factors, averaging and inconsistent ratings. This is historical context, not an authoritative current scoring requirement.

FRX source orientation for Day 2

Review target

Use the learner baseline carried forward from Day 1: **FRX-LEARNER-BASELINE**. Day 2 examines whether implementation and test evidence support the retained requirements and review questions. The source map below is a navigation aid; it is not evidence that a control is present, absent, correct, or complete.

Use only synthetic classroom data and the local learner state. Do not expose the service outside the workstation, use operational credentials, or test a third-party system.

Repository responsibilities

Source	Primary responsibility	Questions it can help answer
frx-project/src/frx_portal/app.py	Creates the Flask application, selects the state module, registers routes, serves the client, and handles selected application errors	Which module supplies route behavior? Which limits and error behavior are application-wide?
frx-project/src/frx_portal/states/vulnerable.py	Implements the learner baseline's route handlers, session checks, database access, upload, analytics, audit, and diagnostic behavior	Where do identity, input, authorization, persistence, output, outbound-call, audit, and failure decisions occur?
frx-project/src/frx_portal/database.py	Defines deterministic users/reports, the SQLite schema, connection lifecycle, initialization, and reset	Which fields represent ownership and assignment? What data persists? Where do transactions begin and end?
frx-project/src/frx_portal/config.py	Defines runtime roots, upload/database/audit locations, analytics origin, and request-size setting	Which values are centralized? Which values constrain files, requests, and dependencies?
frx-project/src/frx_portal/static/client.js	Calls the API and renders the normal browser workflow	Which values originate in the client, which routes receive them, and which responses reach the browser?
frx-project/src/frx_portal/templates/index.html	Defines the learner-visible sign-in, search, report, analytics, and document-intake interface	Which workflows are implemented in the client and which API routes remain outside it?
frx-project/src/frx_portal/fixture_service.py	Implements the deterministic analytics provider	What identity and data does the provider return, and what is outside the portal's direct control?
frx-project/pyproject.toml	Declares Python and package requirements	Which dependencies and versions require later supply-chain evidence?
frx-project/tests/	Characterizes selected application, boundary, and state behavior	Which claims already have tests, which paths are absent, and what would a focused verification need to add?

Route-to-source index

Line numbers refer to the supplied baseline and are starting points, not review conclusions.

Interface	Entry point	Likely next evidence
GET /	app.py:26-28	templates/index.html ; static/client.js
GET /health	states/vulnerable.py:48-50	application creation and selected state
POST /login	states/vulnerable.py:53-73	session configuration, user schema, audit helper, login tests
POST /logout	states/vulnerable.py:76-80	session state and audit evidence
GET /welcome	states/vulnerable.py:83-85	response interpretation and browser context
GET /reports/<report_id>	states/vulnerable.py:88-99	_authenticated , report ownership/assignment fields, audit evidence, allowed and denied-object tests
GET /search	states/vulnerable.py:102-110	caller input, query construction, returned-record authorization, client rendering
POST /documents	states/vulnerable.py:113-127	request limit, file-name/content decisions, upload root, overwrite behavior, audit evidence
POST /analytics/import	states/vulnerable.py:130-150	accepted representation, parser behavior, execution boundary, response/failure tests
GET /analytics/fetch	states/vulnerable.py:153-170	destination parsing, configured origin, provider identity/data, time/size/failure behavior
GET /errors/demo	states/vulnerable.py:173-177	application error handlers, returned fields, audit/log behavior

Day 1 handoff seed

The shared partial chain from Day 1 is review input, not a confirmed implementation finding:

- **T1:** if report entitlement is not enforced on a report-returning path, another partner's report data could be disclosed;
- **R1:** require the trusted service to release report data only when current ownership, assignment, or administrator policy permits it;
- **V1:** observe permitted, denied-object, and no-session behavior without breaking legitimate report access; and
- **Q1:** where and how does every report-returning path enforce the intended access rule?

Unit 1 turns that chain into one selected implementation path. Later units gather source and runtime evidence before assigning finding or checklist status.

Python and Flask construct key

Use this key to read a selected path. It describes what a construct can tell you; it does not establish that the surrounding security behavior is correct.

Construct	Reading meaning	Next question
Application factory, dynamic import, <code>register_blueprint</code>	Selects and assembles the implementation that can receive a request	Which state/module is active, and which handlers or configuration wrap it?
<code>@blueprint.get</code> , <code>@blueprint.post</code> , function signature	Binds an HTTP method/path and converted route values to a Python callable	Which other request or ambient values enter after the function begins?
<code>request.args</code> , <code>request.form</code> , <code>request.get_json</code> , <code>request.files</code> , <code>request.get_data</code>	Reads query, form, JSON, file, or raw-body input	What type, default, size, normalization, validation, and next name apply?
<code>session</code> , <code>current_app</code> , <code>_settings</code>	Reads per-user or application context	Does the value establish identity, authority, configuration, or only availability?
Helper call, <code>if</code> , early <code>return</code> , exception handler	Transfers control or selects an outcome	Which subject, object, action, state, and alternate/failure branch affect the PTH?
<code>connection.execute</code> , <code>fetchone</code> , <code>fetchall</code>	Performs database work and returns rows	How is the statement formed, which values bind, and which rows or fields can reach an effect?
<code>Path.open</code> , uploaded-file <code>save</code>	Reads or writes filesystem state	How are name, destination, content, overwrite, size, and failure behavior decided?
<code>pickletools.genops</code> , JSON parsing, base64 decoding	Interprets a representation using a specific function	Does the exact call inspect, decode, parse, construct, or execute anything?
<code>urllib.request.urlopen</code>	Opens an outbound resource and returns a response stream	Who chooses the destination, what constraints apply, and how are response size/type/failure handled?
<code>jsonify</code> , returned string, response class, template rendering	Creates output in a specific response context	Which fields cross the boundary, what encoding applies, and how do callers consume them?
<code>_audit</code> , <code>app.logger</code> , error handler	Creates an audit/log effect or translates failure	Which event fields, secrets, identifiers, stack details, outcomes, and missing logs matter?
<code>pyproject.toml</code> , imports, tests, fixtures, client calls	Supplies dependency identity, expected behavior, or another side of the path	Is the declared version active, and which promised branches are covered or absent?

Unit 3 source-reading boundary

Unit 3 adds a resumable source trace to one retained PTH and revises CRP1 to revision 0.4. Record registration, exact constructs, call direction, branches, effects, bounded supporting code, the next locator, and

a proof question. Do not create EV records or findings yet; Unit 4 gathers and interprets line-level and runtime evidence.

Evidence discipline

Distinguish these statements in every review record:

- **Known interface:** the route or function exists in the supplied baseline.
- **Source observation:** a named line or function performs an observable operation.
- **Inference:** the reviewer explains what that observation may mean and what would disprove it.
- **Finding:** sufficient evidence demonstrates a weakness in the reviewed baseline.
- **Checklist status:** an authorized rule is applicable and its required evidence has been evaluated.

Do not skip from the first statement to the last two.

Unit 5 current-criteria reference

Use this reference to clarify how direct FRX evidence relates to a governed checklist rule. It does not replace the separately supplied authorized checklist, change a local rule identifier, or assign status without evidence.

Checklist status vocabulary

For this exercise, use the four status values supported by the supplied checklist format and the DISA STIG Viewer 3.x User Guide, Version 1 Release 7 (13 February 2026):

- **Open:** the rule is applicable and sufficient evidence demonstrates that its required condition is not met.
- **Not A Finding:** the rule is applicable and sufficient evidence demonstrates that its required condition is met for the reviewed scope.
- **Not Applicable:** positive scope and feature evidence demonstrates that the rule's condition does not apply; record the rationale and evidence owner.
- **Not Reviewed:** the rule has not been evaluated or the evidence needed for a supported status has not yet been examined.

Do not use an evidence gap as proof of Open, an absence of a search result as proof of Not Applicable, or one passing path as an application-wide Not A Finding.

Current criteria and precise mappings

Review concern	Current verification criterion	Precise weakness when the mechanism is observed	Awareness context only
Database query construction	OWASP ASVS v5.0.0-1.2.4 : verify database queries use parameterization, safe frameworks, or equivalent protection from database injection	CWE-89 for externally influenced input incorporated into an SQL command without correct neutralization	OWASP Top 10 A05:2025 Injection
Data-specific authorization	OWASP ASVS v5.0.0-8.2.2 : verify access to specific data items is restricted to explicitly permitted consumers	CWE-639 when a user-controlled key permits access to another user's record	OWASP Top 10 A01:2025 Broken Access Control
Backend secret handling	OWASP ASVS v5.0.0-13.3.1 : verify backend secrets are securely managed and excluded from source and build artifacts	CWE-798 only when the observed value is a hard-coded credential or cryptographic key; review its mapping cautions	OWASP Top 10 A04:2025 Cryptographic Failures or A07:2025 Authentication Failures only when the mechanism fits
Sensitive logging	OWASP ASVS v5.0.0-16.2.5 : verify logging respects the data's protection level and excludes or masks credentials and similar data	CWE-532 when the product writes sensitive information to a log file	OWASP Top 10 A09:2025 Security Logging and Alerting Failures

Review concern	Current verification criterion	Precise weakness when the mechanism is observed	Awareness context only
Error disclosure	OWASP ASVS <code>v5.0.0-16.5.1</code> : verify unexpected or security-sensitive errors return a generic message without internal data	CWE-209 when an error message contains sensitive information	OWASP Top 10 <code>A10:2025</code> Mishandling of Exceptional Conditions

How to reconcile a local rule with current guidance

1. Preserve the local rule's benchmark ID, rule ID, version, severity, and authorized wording.
2. State the implementation-specific question the rule raises for the selected FRX path.
3. Add the version-qualified current criterion that clarifies the expected control or proof.
4. Record whether the sources align, one is narrower, one is broader, or they conflict.
5. Apply the authorized checklist status to the local rule from direct evidence. Do not assign status to ASVS, CWE, or an OWASP Top 10 category.
6. If the current criterion changes the expected engineering action or exposes ambiguous local wording, record the conflict or proposed deviation and name the approving owner.

Example: a general local special-character rule does not establish one universal treatment. SQL data should remain data through parameterized queries; browser output needs context-appropriate handling. Preserve the local rule, attach the correct current criterion for the observed sink, and record any local-current mismatch instead of inventing an escape-all test.

Sources

- OWASP Application Security Verification Standard 5.0.0, May 2025: <https://owasp.org/www-project-application-security-verification-standard/>
- OWASP SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP Top 10:2025: <https://owasp.org/Top10/2025/>
- MITRE CWE 4.20 entries: <https://cwe.mitre.org/data/definitions/639.html>, <https://cwe.mitre.org/data/definitions/89.html>, <https://cwe.mitre.org/data/definitions/798.html>, <https://cwe.mitre.org/data/definitions/532.html>, and <https://cwe.mitre.org/data/definitions/209.html>
- DISA STIG Viewer 3.x User Guide, Version 1 Release 7, 13 February 2026: https://dl.dod.cyber.mil/wp-content/uploads/stigs/pdf/U_STIG_Viewer_3-x_User_Guide_V1R7.pdf

FRX frozen pipeline evidence bundle

This learner bundle is a deterministic, course-authored training fixture for `FRX-PIPE-006`. It preserves the field shapes and evidence questions learners should recognize in GitLab SAST/secret reports, a CycloneDX SBOM, an OSV-derived vulnerability record, and an SBOM-Researcher-style text report. It is not a live enterprise export and does not assert that the customer's exact rules, versions, thresholds, license policy, or pipeline gates have been approved.

Bound subject

- **Application baseline:** `FRX-LEARNER-BASELINE`, vulnerable learner state.
- **Pipeline evidence set:** `FRX-PIPE-006`, frozen 10 September 2026.
- **Source inputs:** the hashes in `pipeline-manifest.json` bind the claims to the supplied FRX source and dependency files.
- **Network behavior:** none. The bundle is read-only and all vulnerability/advisory data is frozen.

Artifacts

Artifact	What it asserts	What it cannot decide alone
<code>pipeline-manifest.json</code>	Bound source hashes, artifact hashes, tool/profile identity, scan status, scope, and exclusions	Whether any reported mechanism is a finding
<code>gl-sast-report.json</code>	Three static-analysis assertions with rule, location, severity, and proposed mapping	Reachability, business consequence, duplicate identity, or final mapping
<code>gl-secret-detection-report.json</code>	Two pattern matches in source/test material	Whether a value is an active secret or whether two tools found one mechanism
<code>frx.cdx.json</code>	Three identified components and an explicitly incomplete dependency composition	Complete release inventory or license approval
<code>frx_vulns.json</code>	A frozen OSV record for Flask 3.1.2	FRX deployment applicability or exploitability
<code>frx_report.txt</code>	A course-authored SBOM-Researcher-style view of component, advisory, fixed-version, score, and license fields	Organizational risk acceptance, release approval, or a complete component inventory
<code>pipeline-policy.json</code>	The exercise gate threshold and license-owner dependency	Whether a below-threshold signal is false or acceptable

Integrity check

From the Lab 4 folder, run:

```
python inspect-evidence-bundle.py --json
```

The command validates every declared source and artifact hash, prints signal counts, and compares the SBOM component count with the resolved lock. It does not disposition a signal for the learner.

Handling notice

The code and credentials are synthetic classroom material. Do not copy them into another system, scan public targets, or use live credentials. Treat frozen tool assertions as claims that require review.

FRX remediation-review change catalog

Purpose and limits

These are bounded, course-authored excerpts from the proposed FRX `remediated` state. They let a reviewer inspect a correction claim and collect focused proof without receiving the complete instructor reference implementation. Each excerpt must be linked to a confirmed FN record before it enters the learner's remediation queue.

The excerpts do not establish deployment configuration, enterprise identity policy, secret-management policy, monitoring operation, full application coverage, final risk, residual-risk acceptance, or release approval. Use the [Lab 5 guide](#), [record structures](#), and `review-remediation.py` together.

PCH-001 — Runtime session-signing key — worked partial

- **Linked record:** FN1 / SV1 / `SAST-001` / `SECRET-001`.
- **Correction claim:** remove the fixed signing key from source, load a supplied runtime value when present, otherwise generate and retain a separate training-runtime value, and set the supplied cookie flags.
- **Modified surface:** `configure_app`; session configuration and the local runtime key file.

```
session_secret = os.environ.get("FRX_SESSION_SECRET")
if not session_secret:
    if not settings.session_key_path.exists():
        settings.session_key_path.write_text(
            secrets.token_urlsafe(48), encoding="utf-8"
        )
    session_secret = settings.session_key_path.read_text(encoding="utf-8").strip()

app.config.update(
    SECRET_KEY=session_secret,
    SESSION_COOKIE_SECURE=True,
    SESSION_COOKIE_HTTPONLY=True,
    SESSION_COOKIE_SAMESITE="Lax",
)
```

Proof lanes: fixed assignment absent; runtime key created without disclosure; same runtime survives application restart; clean runtimes differ; ordinary login/session works; a request without a session is denied; cookie flags are present.

Required remaining-evidence statement: production secret source, access policy, file permissions/backup, rotation, revocation, previous-session transition, and deployed configuration remain unproved.

PCH-002 — Login query, password storage, and audit handling

- **Possible linked records:** a confirmed weak password-hash or login-query mechanism and related logging evidence.
- **Correction claim:** use a password-hash function intended for password storage, parameterize the username lookup, verify the stored hash, clear prior session state, and keep submitted passwords out of

audit events.

- **Modified surface:** password hashing, login query, login decisions, returned identity, and login audit events.

```
def password_hash(password: str) -> str:
    return generate_password_hash(password, method="scrypt")

user = connection.execute(
    "SELECT id, username, password_hash, role, active FROM users WHERE username = ?",
    (username,),
).fetchone()
if user is None or not user["active"] or not check_password_hash(
    user["password_hash"], password
):
    _audit("login", "failed", username=username)
    return jsonify(error="Invalid username or password"), 401

session.clear()
session.update(user_id=user["id"], username=user["username"], role=user["role"])
```

Proof lanes: ordinary login succeeds; wrong credentials fail generically; punctuation in a missing username does not create a database error; stored hash scheme changes; audit records success/failure without the submitted password; ordinary session behavior remains.

Required remaining-evidence statement: production identity source, migration/forced-reset plan, password policy, rate limits, lockout, and monitoring ownership remain unproved.

PCH-003 — Report object authorization

- **Possible linked record:** CF1/FN authorization mechanism on `GET /reports/<report_id>`.
- **Correction claim:** enforce the stated partner-owner, analyst-assignment, and administrator rules after object lookup and before disclosure; audit both denial and success.
- **Modified surface:** shared `_can_read` decision and report-detail response path.

```
def _can_read(report) -> bool:
    role = session.get("role")
    user_id = session.get("user_id")
    return bool(
        role == "administrator"
        or (role == "partner" and report["owner_id"] == user_id)
        or (role == "analyst" and report["assigned_analyst_id"] == user_id)
    )

if not _can_read(report):
    _audit("report-read", "denied", report_id=report_id, username=session["username"])
    return jsonify(error="Access denied"), 403
```

Proof lanes: anonymous denial; partner own/other records; analyst assigned/unassigned records; administrator access; missing object; denial and success audit events; no weakening of ordinary report reads.

Required remaining-evidence statement: policy-owner confirmation, identity/assignment-data integrity, and consistent enforcement on other interfaces remain unproved.

PCH-004 — Analytics destination and object authorization

- **Possible linked record:** a confirmed or narrowed attacker-influenced analytics-destination mechanism.
- **Correction claim:** accept a positive report identifier rather than a caller URL, authorize the referenced report, construct the destination from configured service origin plus application-selected path, validate response identity, and return bounded failures.
- **Modified surface:** `GET /analytics/fetch`, report lookup/authorization, outbound construction, response validation, and error handling.

```
report_id = request.args.get("report_id", type=int)
if report_id is None or report_id < 1:
    return jsonify(error="A positive report_id is required"), 400

if not _can_read(report):
    return jsonify(error="Access denied"), 403

destination = f"{_settings().analytics_base_url}/metrics/{report_id}"
with urllib.request.urlopen(destination, timeout=2) as remote:
    payload = remote.read(64 * 1024)

decoded = json.loads(payload)
if not isinstance(decoded, dict) or decoded.get("report_id") != report_id:
    return jsonify(error="Analytics response failed validation"), 502
```

Proof lanes: caller URL parameter rejected; positive owned identifier succeeds; other-owner identifier denied; invalid identifier and provider failures are bounded; only the configured origin/application path is called; response identifier must match.

Required remaining-evidence statement: production service identity, TLS/DNS/proxy/egress controls, provider authenticity/availability, and other outbound interfaces remain unproved.

PCH-005 — Audit and error-output correction — extension

- **Possible linked records:** confirmed secret-bearing audit or sensitive-error mechanisms.
- **Correction claim:** use structured event/outcome fields without submitted secrets and return a generic handled error while retaining an audit event.
- **Modified surface:** audit helper, login audit calls, demonstration error response, and global error behavior.

```
def _audit(event: str, outcome: str, **fields) -> None:
    safe_fields = " ".join(f"{key}={value}" for key, value in sorted(fields.items()))
    with _settings().audit_log_path.open("a", encoding="utf-8") as stream:
        stream.write(f"event={event} outcome={outcome} {safe_fields}".rstrip() + "\n")

_audit("demo-error", "handled", username=session["username"])
return jsonify(error="The requested training resource is unavailable."), 503
```

Proof lanes: authentication success/failure remains recorded; submitted password and configured key are absent; internal path is absent from the response; client failure is bounded; repository tests remain green.

Required remaining-evidence statement: central collection, access control, retention, tamper protection, alert routing, and deployed unhandled-exception behavior remain unproved.